

Distributed Data Processing with PySpark

Technical Notes

Rodrigo Kang

Table of contents

1	Distributed Data Processing	8
2	Why Spark?	9
2.1	Scaling Up and Scaling Out	10
3	What Is PySpark?	10
4	When Is PySpark Useful?	11
4.1	Data Exceeds Convenient Single-Machine Capacity	11
4.2	Computational Work Can Be Distributed	12
4.3	Data Pipelines Operate at Large Scale	12
4.4	The Data Already Lives in a Distributed Environment	12
5	When Is PySpark Not Necessary?	13
6	The Spark Computational Model	14
6.1	Driver	14
6.2	Executors	15
6.3	Cluster Manager	15
6.4	Tasks	16
7	Partitions	16
8	Local Mode and Cluster Mode	17
9	SparkSession	18
10	Spark DataFrames	19
11	DataFrames and pandas DataFrames	20
12	Lazy Evaluation	21
13	Transformations	22

14 Actions	23
15 Why Lazy Evaluation Matters	24
16 Execution Plans	25
17 Creating Spark DataFrames	26
18 Schemas	27
18.1 Schema Inference	28
18.2 Explicit Schemas	29
19 Inspecting a DataFrame	30
20 Selecting Columns	31
21 Column Expressions	33
22 Adding and Replacing Columns	34
23 Renaming and Dropping Columns	35
24 Filtering Rows	36
25 Working with Strings	38
26 Conditional Expressions	38
27 Missing Values	39
28 Distinct Values and Duplicates	41
29 Sorting	41
30 Aggregation	42
31 Grouped Aggregations	44
32 Local Computation and Data Movement	45
33 Narrow and Wide Transformations	46
34 Stages	47
35 Grouping and Shuffle Cost	48
36 Working with Multiple Aggregations	49
37 Counting Distinct Values	50
38 Basic Descriptive Statistics	51

39 Collecting Data to the Driver	51
40 Converting to pandas	53
41 DataFrame API as a Declarative Interface	53
42 Joining DataFrames	55
43 Join Types	57
43.1 Inner Join	57
43.2 Left Join	58
43.3 Full Join	58
44 Semi and Anti Joins	58
45 Joining with Explicit Conditions	59
46 Joins and Row Multiplication	60
47 Why Joins Can Be Expensive	61
48 Shuffle Joins	62
49 Broadcast Joins	63
50 When Broadcasting Helps	64
51 Join Strategy versus Join Semantics	65
52 Partitioning	66
53 Repartitioning	67
54 Increasing and Decreasing Partitions	67
55 Partition Balance	69
56 Data Skew	69
57 More Workers Do Not Guarantee Linear Speedup	70
58 Repeated Computation	71
59 Caching	72
60 Persistence	73
61 When to Cache	73
62 Removing Cached Data	74

63 Lineage	74
64 Reading an Execution Plan	75
65 Recognising Exchange	76
66 Partial Aggregation	77
67 Logical and Physical Plans	77
68 Query Optimisation	78
69 Predicate Pushdown	79
70 Column Pruning	80
71 Adaptive Query Execution	80
72 Data Skew and Adaptive Execution	81
73 Built-In Functions versus Python UDFs	82
74 User-Defined Functions	83
75 Performance Reasoning	83
76 A Simple Optimisation Example	84
77 Spark Performance as Data-Movement Management	86
78 A Practical Mental Model	87
79 Spark SQL	88
80 Temporary Views	89
81 DataFrame API versus SQL	90
82 SQL Joins	91
83 Window Functions	92
84 Defining a Window	93
85 Ordered Windows	94
86 Row Number	94
87 Ranking	95
88 Previous and Next Observations	96

89	Running Aggregations	97
90	Window Functions and Performance	98
91	Writing Data	99
92	Writing CSV	99
93	Why Spark Produces Multiple Files	100
94	Parquet	101
95	Why Parquet Is Often Preferable to CSV	102
96	Partitioned Output	103
97	Choosing Partition Columns	104
98	The Small Files Problem	104
99	Spark and pandas	105
100	Moving from Spark to pandas	106
101	Apache Arrow	106
102	Pandas UDFs	107
103	Built-In Expressions First	108
104	Machine Learning with Spark	109
105	Estimator and Transformer	110
106	Feature Vectors	110
107	A Simple Spark ML Estimator	111
108	Spark ML Pipelines	112
109	PySpark and Large-Scale Machine Learning	114
110	Spark versus pandas for Machine Learning Preparation	114
111	Data Leakage in Spark Workflows	115
112	Common PySpark Mistakes	116
112.1	Treating Spark Like pandas	116
112.2	Calling collect() Too Early	117
112.3	Calling toPandas() on Large Data	117
112.4	Excessive Use of Python UDFs	117

112.5	Unnecessary Shuffles	117
112.6	Repartitioning Without a Reason	118
112.7	Caching Everything	118
112.8	Ignoring Skew	118
112.9	Producing Too Many Small Files	118
112.10	Ignoring the Execution Plan	119
113A	Practical Spark Workflow	119
114	PySpark in a Data Science Workflow	120
115	Spark as an Execution Engine	121
116	Choosing Spark	121
117	Key Takeaways	122
118	Problems	124
118.1	Problem 1 — Creating a Spark Session	124
118.2	Problem 2 — Creating a DataFrame	125
118.3	Problem 3 — Explicit Schema	125
118.4	Problem 4 — Basic Inspection	126
118.5	Problem 5 — Selecting Columns	126
118.6	Problem 6 — Column Expressions	127
118.7	Problem 7 — Multiple Derived Columns	127
118.8	Problem 8 — Filtering Rows	128
118.9	Problem 9 — Conditional Columns	128
118.10	Problem 10 — String Operations	129
118.11	Problem 11 — Missing Values	129
118.12	Problem 12 — Duplicates	130
118.13	Problem 13 — Sorting	130
118.14	Problem 14 — Global Aggregation	130
118.15	Problem 15 — Grouped Aggregation	131
118.16	Problem 16 — Grain after Aggregation	132
118.17	Problem 17 — Transformations and Actions	132
118.18	Problem 18 — Lazy Evaluation	132
118.19	Problem 19 — Reading an Execution Pipeline	133
118.20	Problem 20 — Narrow versus Wide Transformations	134
118.21	Problem 21 — Shuffle Reasoning	134
118.22	Problem 22 — Stages	135
118.23	Problem 23 — Basic Join	135
118.24	Problem 24 — Join Types	136
118.25	Problem 25 — Semi Join	137
118.26	Problem 26 — Anti Join	137
118.27	Problem 27 — Join Cardinality	137
118.28	Problem 28 — Many-to-Many Row Multiplication	137
118.29	Problem 29 — Join and Shuffle	138
118.30	Problem 30 — Broadcast Join	138

118.3	Problem 31 — When Not to Broadcast	139
118.3	Problem 32 — Logical Join versus Physical Strategy	139
118.3	Problem 33 — Number of Partitions	139
118.3	Problem 34 — repartition() versus coalesce()	140
118.3	Problem 35 — Repartitioning by Key	140
118.3	Problem 36 — Too Few Partitions	141
118.3	Problem 37 — Too Many Partitions	141
118.3	Problem 38 — Data Skew	141
118.3	Problem 39 — Straggler Task	142
118.4	Problem 40 — More Executors	142
118.4	Problem 41 — Repeated Computation	143
118.4	Problem 42 — Cache	143
118.4	Problem 43 — When Not to Cache	143
118.4	Problem 44 — collect()	144
118.4	Problem 45 — toPandas()	144
118.4	Problem 46 — Reading an Execution Plan	145
118.4	Problem 47 — Filter Early	145
118.4	Problem 48 — Column Pruning	145
118.4	Problem 49 — Built-In Function versus UDF	146
118.5	Problem 50 — Appropriate Use of a Python UDF	146
118.5	Problem 51 — Spark SQL	147
118.5	Problem 52 — SQL Join	147
118.5	Problem 53 — Window versus Aggregation	147
118.5	Problem 54 — Row Number	148
118.5	Problem 55 — Ranking	148
118.5	Problem 56 — Lag	148
118.5	Problem 57 — Running Total	149
118.5	Problem 58 — Window Performance	149
118.5	Problem 59 — Writing Parquet	150
118.6	Problem 60 — CSV versus Parquet	150
118.6	Problem 61 — Partitioned Output	150
118.6	Problem 62 — Bad Partition Column	151
118.6	Problem 63 — Small Files Problem	151
118.6	Problem 64 — Arrow	151
118.6	Problem 65 — Pandas UDF	151
118.6	Problem 66 — PySpark or pandas?	152
118.6	Problem 67 — Building a Modelling Table	152
118.6	Problem 68 — Leakage in Distributed Feature Engineering	153
118.6	Problem 69 — VectorAssembler	153
118.7	Problem 70 — Estimator and Model	153
118.7	Problem 71 — Spark ML Pipeline	154
118.7	Problem 72 — Distributed Data Preparation versus Distributed Training	154
118.7	Problem 73 — Diagnose a Poor Spark Workflow	155
118.7	Problem 74 — Diagnose Repeated Actions	155
118.7	Problem 75 — Diagnose Unnecessary Repartitioning	156
118.7	Problem 76 — Diagnose a Join Strategy	156

118.7	Problem 77 — Diagnose Skew	156
118.7	Problem 78 — Execution Plan Reasoning	157
118.7	Problem 79 — End-to-End Spark Reasoning	157
118.8	Problem 80 — Complete Architecture Exercise	159

1 Distributed Data Processing

Many data-processing tasks can be performed efficiently on a single computer.

A dataset can be loaded into memory, transformed using libraries such as NumPy or pandas, analysed, and used to train statistical or machine learning models. For a large class of practical problems, this is the simplest and most appropriate computational model.

However, the single-machine approach eventually encounters physical limits.

A dataset may become too large to fit comfortably in memory. A transformation may require more CPU resources than one machine can provide within an acceptable amount of time. Data may already be distributed across many files or storage locations. A production pipeline may need to process substantially more information than a local analytical workflow.

At that point, increasing the resources of a single machine is not always sufficient or economically attractive.

An alternative is **distributed computing**: dividing a computational problem across multiple machines and coordinating their work.

Conceptually, instead of asking one machine to process an entire dataset,

$$D,$$

the dataset can be divided into partitions,

$$D = D_1 \cup D_2 \cup \dots \cup D_p,$$

where different partitions can be processed by different workers.

For an operation that can be applied independently to each partition,

$$f(D),$$

the computation may be decomposed approximately as

$$f(D) \longrightarrow \{f(D_1), f(D_2), \dots, f(D_p)\}.$$

The partial results can then be combined when required.

This apparently simple idea introduces a substantially different computational model.

Once several machines participate in the same computation, the system must address questions such as:

- how the data is partitioned;
- where each computation is executed;
- how intermediate results are transferred between machines;
- how tasks are scheduled;
- what happens when a task or machine fails;
- how dependencies between computations are represented;
- how communication costs are minimised.

Distributed data processing is therefore not simply ordinary Python executed on a larger computer. It requires an execution engine capable of coordinating computation across a cluster.

2 Why Spark?

Apache Spark is a distributed computing engine designed for large-scale data processing.

Spark allows computations to be expressed at a relatively high level while the engine manages much of the complexity associated with distributed execution.

Rather than manually deciding which machine should process each part of a dataset, the user describes a sequence of transformations. Spark determines how those transformations should be organised and executed across the available resources.

A simplified view is

data $\longrightarrow$ transformations $\longrightarrow$ execution plan $\longrightarrow$ distributed computation.

This separation between the **logical description of a computation** and its **physical execution** is one of the central ideas behind Spark.

2.1 Scaling Up and Scaling Out

Two broad strategies can be used when a computation requires more resources.

The first is **vertical scaling**, or **scaling up**:

larger machine = more memory + more CPU.

For example, a workload running out of memory on a machine with 16 GB of RAM might be moved to a machine with 64 GB or 128 GB.

Vertical scaling is often the simplest solution and should not be dismissed. If a problem fits comfortably on one machine, introducing distributed infrastructure may create unnecessary complexity.

The second strategy is **horizontal scaling**, or **scaling out**:

more computational capacity = multiple cooperating machines.

Instead of relying on one increasingly powerful machine, the workload is distributed across a cluster.

Spark is primarily designed around this second model.

The distinction is important because distributed computing introduces costs that do not exist in ordinary local computation. Machines must communicate, data may need to move across a network, tasks must be coordinated, and failures must be handled.

Consequently,

distributed computing is not automatically faster than single-machine computing.

For small datasets, pandas or NumPy will often be simpler and may also be faster. Spark becomes useful when the characteristics of the workload justify the overhead of distributed execution.

3 What Is PySpark?

Spark itself is implemented primarily on the Java Virtual Machine. **PySpark** provides the Python interface to Apache Spark.

It allows Spark computations to be expressed using Python while the underlying Spark engine coordinates their execution.

Conceptually,

Python code $\longrightarrow$ PySpark API $\longrightarrow$ Spark execution engine $\longrightarrow$ cluster resources.

This makes Spark particularly relevant to data scientists and machine learning practitioners who already work primarily in Python.

A PySpark workflow can therefore resemble familiar Python data-processing code:

```
1 (
2   df
3   .filter(...)
4   .groupBy(...)
5   .agg(...)
6 )
```

but the computational semantics are different from those of a pandas DataFrame.

A pandas DataFrame normally represents data manipulated directly by the current Python process. A Spark DataFrame represents a **distributed dataset together with a logical computation that Spark can plan and execute across partitions**.

This distinction becomes fundamental when reasoning about performance.

4 When Is PySpark Useful?

PySpark is particularly useful when one or more of the following conditions apply.

4.1 Data Exceeds Convenient Single-Machine Capacity

Suppose a dataset requires substantially more memory than is available on the analytical machine.

A pandas workflow generally expects the working data to fit within the memory constraints of a single process and machine.

Spark can distribute partitions of the data across several workers, allowing the total dataset to exceed the memory capacity of any individual worker.

This does not mean that Spark removes memory constraints. Each executor still has finite memory. Rather, the workload is distributed so that no single machine necessarily needs to contain the entire dataset.

4.2 Computational Work Can Be Distributed

Some operations can be applied independently to different portions of a dataset.

For example, consider transforming every observation:

$$x_i \longrightarrow f(x_i).$$

If observations can be processed independently, different partitions can often be transformed concurrently.

Distributed execution can therefore increase available computational throughput.

4.3 Data Pipelines Operate at Large Scale

Spark is frequently used not only for exploratory analysis but for data-engineering and production workloads involving:

- large collections of files;
- distributed object storage;
- data lakes;
- ETL and ELT pipelines;
- large joins and aggregations;
- feature engineering;
- repeated batch-processing jobs.

The value of Spark in such settings comes from both its scale and its integration with distributed data infrastructure.

4.4 The Data Already Lives in a Distributed Environment

Sometimes the motivation for Spark is not simply dataset size.

If data already resides in distributed storage and downstream systems operate in a distributed environment, moving all data to one machine merely to process it locally may be inefficient.

In such situations it can be preferable to **move computation towards the data** rather than repeatedly moving large quantities of data towards a single Python process.

5 When Is PySpark Not Necessary?

Understanding when *not* to use Spark is as important as knowing how to use it.

Spark introduces:

- cluster-management overhead;
- task-scheduling overhead;
- serialisation costs;
- communication between workers;
- potentially expensive data movement;
- additional operational complexity.

For a dataset that fits comfortably into memory on one machine, code such as

```
1 import pandas as pd
2
3 df = pd.read_csv(
4     "data.csv"
5 )
6
7 result = (
8     df
9     .groupby("category")["sales"]
10    .mean()
11 )
```

may be entirely sufficient.

Rewriting the same small computation in Spark does not inherently make it more scalable in a useful sense. It may simply introduce distributed-system overhead into a problem that did not require distributed computing.

A useful engineering principle is therefore:

use distributed computing when the scale or architecture of the problem requires it,
not merely because distributed tools are available.

This principle is particularly relevant in data science. The objective is not to use the most sophisticated infrastructure possible, but to choose an appropriate computational model for the workload.

6 The Spark Computational Model

A Spark application involves several interacting components.

At a high level, the architecture can be represented as



The main concepts are:

- **driver;**
- **cluster manager;**
- **executors;**
- **tasks;**
- **partitions.**

Understanding these components is more important than memorising individual PySpark methods because they explain how apparently simple DataFrame operations become distributed computations.

6.1 Driver

The **driver** is the process that coordinates the Spark application.

It is responsible for tasks such as:

- running the main application logic;
- creating the Spark session;
- constructing logical execution plans;
- coordinating work;
- requesting resources;

- scheduling distributed computation;
- collecting results when necessary.

When working interactively with PySpark, the code written by the user is conceptually being coordinated from the driver.

The driver should therefore not be confused with the cluster as a whole.

6.2 Executors

Executors are processes responsible for performing work on behalf of a Spark application.

Executors:

- execute tasks;
- process partitions of data;
- store cached data;
- produce intermediate results;
- communicate results and status back to the driver.

If a dataset is divided into many partitions, those partitions can be processed by tasks running across available executors.

6.3 Cluster Manager

The **cluster manager** allocates computational resources to Spark applications.

Depending on the environment, Spark can operate with different resource-management systems.

The important conceptual separation is:

Spark = computation engine,

whereas the cluster manager is responsible for allocating the resources on which that computation runs.

This distinction becomes especially relevant in production and cloud environments.

6.4 Tasks

A **task** is a unit of work executed on a partition.

If an operation must be applied to a dataset containing many partitions, Spark can create tasks that process those partitions across available executor resources.

Very approximately,

$$\text{stage} \longrightarrow \text{tasks} \longrightarrow \text{partitions}.$$

A partition is therefore closely related to the unit of parallelism in Spark.

7 Partitions

A Spark dataset is divided into **partitions**.

Suppose a dataset contains N observations and is divided into p partitions:

$$D = D_1 \cup D_2 \cup \dots \cup D_p.$$

A worker does not normally process the abstract dataset D all at once. Tasks operate on individual partitions.

This has an important consequence:

Spark parallelism operates primarily through partitions.

If a large dataset has too few partitions, the cluster may be underutilised because there are not enough independent units of work.

If it has excessively many very small partitions, scheduling overhead can become significant.

Partitioning therefore influences:

- parallelism;
- memory usage;
- network communication;
- task scheduling;
- overall execution performance.

The number and distribution of partitions are not merely implementation details. They are part of the computational structure of a Spark workload.

8 Local Mode and Cluster Mode

Spark does not require a physical multi-machine cluster for every execution.

It can also run in **local mode**, where the Spark components execute on a single machine.

This is useful for:

- learning;
- development;
- testing;
- debugging;
- small workloads.

For example, a local Spark session can be created with:

```
1 from pyspark.sql import SparkSession
2
3 spark = (
4     SparkSession
5     .builder
6     .master("local[*]")
7     .appName("PySparkTechnicalNotes")
8     .getOrCreate()
9 )
```

The expression

```
1 local[*]
```

instructs Spark to run locally and use the available processor cores.

The important point is that the programming model remains Spark's distributed programming model even though the execution environment is local.

This allows code to be developed on a workstation and later executed in a distributed environment with relatively little change to the high-level DataFrame logic.

9 SparkSession

The main entry point for modern PySpark applications is the SparkSession.

```
1 from pyspark.sql import SparkSession
2
3 spark = (
4     SparkSession
5     .builder
6     .appName("PySparkTechnicalNotes")
7     .getOrCreate()
8 )
```

The resulting object:

```
1 spark
```

provides access to functionality for:

- creating DataFrames;
- reading data;
- executing SQL;
- configuring Spark;
- interacting with the underlying Spark environment.

For example, a small DataFrame can be created from Python data:

```
1 data = [
2     ("Auckland", 1_440_300),
3     ("Christchurch", 396_200),
4     ("Wellington", 202_700)
5 ]
6
7 columns = [
8     "city",
9     "population"
10 ]
11
12 df = spark.createDataFrame(
13     data,
14     columns
```

```
15 )  
16  
17 df.show()
```

Although this example contains only a few rows, `df` is a Spark `DataFrame` and therefore follows Spark's execution model.

The significance of that model becomes clearer once transformations, actions, and lazy evaluation are introduced.

10 Spark DataFrames

The **DataFrame** is the central high-level abstraction used in modern PySpark data processing.

A Spark `DataFrame` represents distributed tabular data organised into named columns.

Conceptually, it resembles a table:

city	population
Auckland	1440300
Christchurch	396200
Wellington	202700

but its implementation and execution semantics differ substantially from a pandas `DataFrame`.

A Spark `DataFrame` has a **schema** describing the name and data type of each column.

The schema can be inspected using:

```
1 df.printSchema()
```

and a sample of the data can be displayed with:

```
1 df.show()
```

The column names can be obtained with:

```
1 df.columns
```

while:

```
1 df.schema
```

provides the complete structural description.

For example:

```
1 df.printSchema()
```

may produce a structure conceptually similar to:

```
1 root
2 |-- city: string
3 |-- population: long
```

Schemas matter in distributed processing because Spark needs information about data types in order to plan and optimise computations.

11 DataFrames and pandas DataFrames

The visual similarity between pandas and Spark DataFrames can obscure an important difference.

With pandas:

```
1 result = (
2     df
3     .groupby("category")
4     ["sales"]
5     .mean()
6 )
```

the operations are normally executed directly by the local Python process and its underlying libraries.

With Spark:

```

1 result = (
2     df
3     .groupBy("category")
4     .avg("sales")
5 )

```

the expression contributes to a **logical execution plan**.

Spark can inspect this plan, optimise it, determine how data should be distributed, and execute the required tasks across partitions.

Thus, a useful mental distinction is:

pandas DataFrame $\approx$ local in-memory data structure,

whereas

Spark DataFrame $\approx$ distributed data abstraction + execution plan.

The distinction is not absolute in every implementation detail, but it provides a much better mental model for reasoning about Spark.

12 Lazy Evaluation

One of the most important features of Spark is **lazy evaluation**.

Many Spark operations do not immediately perform the requested computation.

Consider:

```

1 filtered = df.filter(
2     df.population > 300_000
3 )

```

At this point, Spark does not necessarily scan the entire dataset and materialise the filtered result.

Instead, the transformation is recorded as part of a logical plan.

Another transformation can be added:

```
1 selected = filtered.select(  
2     "city"  
3 )
```

Again, the complete distributed computation need not execute immediately.

Conceptually, Spark builds a sequence such as:

$$D \longrightarrow \text{filter} \longrightarrow \text{select}.$$

Only when a result is actually required does Spark execute the necessary computation.

For example:

```
1 selected.show()
```

requires data to be produced and therefore triggers execution.

This distinction leads to two fundamental categories of Spark operations:

- **transformations;**
- **actions.**

13 Transformations

A **transformation** describes how one distributed dataset should be transformed into another.

Examples include:

```
1 df.select(...)
```

```
1 df.filter(...)
```

```
1 df.withColumn(...)
```

```
1 df.groupBy(...)
```

```
1 df.join(...)
```

Transformations are generally evaluated lazily.

For example:

```
1 adults = (  
2     people  
3     .filter(  
4         people.age >= 18  
5     )  
6     .select(  
7         "name",  
8         "age"  
9     )  
10 )
```

defines a transformation pipeline but does not necessarily execute it immediately.

Conceptually,

$$D \xrightarrow{\text{filter}} D' \xrightarrow{\text{select}} D''.$$

The resulting DataFrame is another logical distributed dataset.

14 Actions

An **action** requests a concrete result and therefore triggers execution of the required transformations.

Common actions include:

```
1 df.show()
```

```
1 df.count()
```

```
1 df.collect()
```

```
1 df.take(10)
```

For example:

```
1 adults.count()
```

requires Spark to determine how many observations satisfy the preceding transformation pipeline.

Spark can then construct and execute the required physical plan.

The distinction can be summarised as:

transformations $\rightarrow$ build computation,

action $\rightarrow$ execute computation.

15 Why Lazy Evaluation Matters

Lazy evaluation gives Spark an opportunity to reason about the computation before executing it.

Suppose the user writes:

```
1 result = (  
2   df  
3   .filter(  
4     df.country == "New Zealand"  
5   )  
6   .select(  
7     "customer_id",  
8     "sales"  
9   )  
10 )
```

If each operation were executed immediately, Spark would have less opportunity to optimise the computation globally.

Instead, Spark can analyse the complete logical sequence and determine an efficient execution strategy.

This is similar in spirit to the way a relational database can optimise a SQL query before executing it.

The user therefore specifies primarily **what transformations are required**, while Spark retains substantial control over **how those transformations are physically executed**.

This declarative character is one reason the DataFrame API is central to modern Spark.

16 Execution Plans

Spark DataFrame operations produce execution plans that can be inspected.

For example:

```
1 result.explain()
```

shows information about how Spark intends to execute the computation.

A useful conceptual hierarchy is:

user expression → logical plan → optimised logical plan → physical plan → tasks.

This is a major difference from thinking about Python code purely as a sequence of immediately executed statements.

When working with Spark, performance problems are often better understood by asking:

- what execution plan did Spark construct?
- how is the data partitioned?
- does the operation require data movement?
- how many tasks are being created?
- is the same computation being repeated?

rather than simply asking whether an individual Python expression appears computationally expensive.

This shift in mental model is fundamental to effective Spark programming.

17 Creating Spark DataFrames

Spark DataFrames can be created from several sources.

For small examples, Python objects can be converted directly into a DataFrame:

```
1 data = [  
2     (1, "Auckland", 120.0),  
3     (2, "Wellington", 95.0),  
4     (3, "Christchurch", 140.0),  
5     (4, "Auckland", 110.0)  
6 ]  
7  
8 columns = [  
9     "customer_id",  
10    "city",  
11    "sales"  
12 ]  
13  
14 df = spark.createDataFrame(  
15     data,  
16     columns  
17 )  
18  
19 df.show()
```

This is convenient for demonstrations and tests, but production Spark workloads more commonly read data from external storage.

Spark supports several data formats, including:

- CSV;
- JSON;
- Parquet;
- ORC;
- tables exposed through catalogues or metastore systems.

The general pattern is:

```
1 df = (  
2     spark  
3     .read
```

```
4 .format(...)
5 .option(...)
6 .load(...)
7 )
```

Convenience methods are also available for common formats.

For example:

```
1 df = (
2     spark
3     .read
4     .option(
5         "header",
6         True
7     )
8     .option(
9         "inferSchema",
10        True
11    )
12    .csv(
13        "data/customers.csv"
14    )
15 )
```

A Parquet dataset can be read more directly:

```
1 df = spark.read.parquet(
2     "data/customers.parquet"
3 )
```

The resulting object is a Spark DataFrame regardless of the underlying storage format.

18 Schemas

A Spark DataFrame has a **schema** describing its columns and their data types.

The schema can be inspected with:

```
1 df.printSchema()
```

or accessed programmatically:

```
1 df.schema
```

For example, a schema might conceptually contain:

```
1 root
2 |-- customer_id: long
3 |-- city: string
4 |-- sales: double
```

Data types are important because Spark uses schema information when analysing and planning computations.

18.1 Schema Inference

Spark can sometimes infer a schema from the input data.

For example:

```
1 df = (
2     spark
3     .read
4     .option(
5         "header",
6         True
7     )
8     .option(
9         "inferSchema",
10        True
11    )
12    .csv(
13        "data/customers.csv"
14    )
15 )
```

This is convenient during exploration.

However, schema inference requires Spark to inspect the data and may also produce types that differ from the intended data model.

For controlled production pipelines, it is generally preferable to define the schema explicitly.

18.2 Explicit Schemas

A schema can be constructed using types from:

```
1 pyspark.sql.types
```

For example:

```
1 from pyspark.sql.types import (  
2     StructType,  
3     StructField,  
4     IntegerType,  
5     StringType,  
6     DoubleType  
7 )  
8  
9 schema = StructType([  
10     StructField(  
11         "customer_id",  
12         IntegerType(),  
13         False  
14     ),  
15     StructField(  
16         "city",  
17         StringType(),  
18         True  
19     ),  
20     StructField(  
21         "sales",  
22         DoubleType(),  
23         True  
24     )  
25 ])
```

The Boolean argument in each StructField indicates whether the column is nullable.

The schema can then be supplied when reading the data:

```
1 df = (  
2     spark  
3     .read  
4     .option(  
5         "header",  
6         True  
7     )  
8     .schema(schema)  
9     .csv(  
10        "data/customers.csv"  
11    )  
12 )
```

An explicit schema provides a clear contract between the input data and the processing pipeline.

It can also avoid the additional work required for schema inference.

19 Inspecting a DataFrame

Several methods are useful when initially inspecting a Spark DataFrame.

The column names are available through:

```
1 df.columns
```

The schema can be displayed with:

```
1 df.printSchema()
```

A sample can be displayed with:

```
1 df.show()
```

For example:

```
1 df.show(  
2     5,  
3     truncate=False  
4 )
```

shows up to five rows without truncating long values.

The number of observations can be obtained with:

```
1 df.count()
```

However, there is an important difference between expressions such as:

```
1 df.columns
```

and:

```
1 df.count()
```

Accessing column metadata does not require Spark to count every observation in the distributed dataset.

By contrast:

```
1 df.count()
```

is an **action** and therefore requires execution of the relevant Spark plan.

This distinction matters when working with large datasets. Operations that appear trivial from the Python interface may require substantial distributed computation.

20 Selecting Columns

Columns can be selected using:

```
1 select()
```

For example:

```
1 selected = df.select(  
2     "customer_id",  
3     "sales"  
4 )
```

A column can also be represented explicitly using:

```
1 col()
```

from:

```
1 pyspark.sql.functions
```

For example:

```
1 from pyspark.sql import functions as F  
2  
3 selected = df.select(  
4     F.col("customer_id"),  
5     F.col("sales")  
6 )
```

Using:

```
1 F.col(...)
```

is particularly useful when building more complex expressions.

For example:

```
1 df.select(  
2     F.col("sales"),  
3     (  
4         F.col("sales") * 1.15  
5     ).alias("sales_adjusted")  
6 )
```

Here,


```
1 F.col("sales")
```

does not extract the column into a local Python object.

Instead, it constructs a **Spark column expression** that becomes part of the DataFrame's logical plan.

This distinction is important.

Spark expressions should generally remain inside the Spark execution model rather than unnecessarily moving data into ordinary Python objects.

21 Column Expressions

PySpark DataFrame operations are built extensively from column expressions.

Consider:

```
1 F.col("sales") * 1.15
```

This does not immediately multiply every value.

It represents an expression that Spark can incorporate into its execution plan.

Expressions can be combined:

```
1 (
2   F.col("quantity")
3   * F.col("unit_price")
4 )
```

and named using:

```
1 alias()
```

For example:

```

1 df.select(
2     "customer_id",
3     (
4         F.col("quantity")
5         * F.col("unit_price")
6     ).alias("revenue")
7 )

```

This declarative style is central to the DataFrame API.

Instead of manually iterating through observations, the user describes transformations over columns and Spark determines how to execute them.

22 Adding and Replacing Columns

New columns can be created using:

```

1 withColumn()

```

For example:

```

1 df = df.withColumn(
2     "sales_tax",
3     F.col("sales") * 0.15
4 )

```

Another derived column can then be created:

```

1 df = df.withColumn(
2     "sales_with_tax",
3     F.col("sales")
4     + F.col("sales_tax")
5 )

```

If the specified column name already exists:

```

1 withColumn()

```

replaces that column.

For example:

```
1 df = df.withColumn(  
2     "sales",  
3     F.col("sales").cast("double")  
4 )
```

changes the expression associated with the sales column.

Spark DataFrames should generally be treated as **immutable transformations**.

The expression:

```
1 df.withColumn(...)
```

does not mutate the existing DataFrame in place in the conventional Python sense.

Instead, it returns another DataFrame representing a new logical transformation.

Thus:

```
1 df2 = df.withColumn(  
2     "sales_tax",  
3     F.col("sales") * 0.15  
4 )
```

can be interpreted conceptually as:

$$D_2 = T(D_1),$$

where T is the transformation that adds the derived column.

23 Renaming and Dropping Columns

Columns can be renamed using:

```
1 withColumnRenamed()
```

For example:

```
1 df = df.withColumnRenamed(  
2     "sales",  
3     "total_sales"  
4 )
```

Columns can be removed using:

```
1 drop()
```

For example:

```
1 df = df.drop(  
2     "temporary_column"  
3 )
```

Several columns can be removed together:

```
1 df = df.drop(  
2     "column_a",  
3     "column_b"  
4 )
```

24 Filtering Rows

Rows can be filtered using:

```
1 filter()
```

or its alias:

```
1 where()
```

For example:

```
1 high_sales = df.filter(  
2     F.col("sales") > 100  
3 )
```

Multiple conditions can be combined:

```
1 result = df.filter(  
2     (F.col("sales") > 100)  
3     & (F.col("city") == "Auckland")  
4 )
```

The Boolean operators commonly used with Spark column expressions are:

```
1 &  AND  
2 |  OR  
3 ~  NOT
```

Parentheses are important when combining conditions.

For example:

```
1 result = df.filter(  
2     (  
3         F.col("city")  
4         == "Auckland"  
5     )  
6     & (  
7         F.col("sales")  
8         >= 100  
9     )  
10 )
```

This expression contributes a filter operation to the logical plan.

Execution is still lazy until an action is requested.

For example:

```
1 result.show()
```

triggers the computation.

25 Working with Strings

PySpark provides many built-in functions for string manipulation.

For example:

```
1 df.select(  
2     F.upper(  
3         F.col("city")  
4     ).alias("city_upper")  
5 )
```

Other common operations include:

```
1 F.lower(...)  
2 F.trim(...)  
3 F.length(...)  
4 F.substring(...)  
5 F.concat(...)  
6 F.concat_ws(...)
```

For example:

```
1 df = df.withColumn(  
2     "city_normalised",  
3     F.lower(  
4         F.trim(  
5             F.col("city")  
6         )  
7     )  
8 )
```

Built-in Spark functions are generally preferable to implementing equivalent logic manually in Python because they remain within Spark's optimisable execution model.

26 Conditional Expressions

Conditional logic can be expressed using:

```
1 when()
```

and:

```
1 otherwise()
```

For example:

```
1 df = df.withColumn(  
2     "sales_category",  
3     F.when(  
4         F.col("sales") >= 120,  
5         "high"  
6     )  
7     .when(  
8         F.col("sales") >= 100,  
9         "medium"  
10    )  
11    .otherwise(  
12        "low"  
13    )  
14 )
```

This is conceptually similar to:

$$c(x) = \begin{cases} \text{high}, & x \geq 120, \\ \text{medium}, & 100 \leq x < 120, \\ \text{low}, & x < 100. \end{cases}$$

The resulting conditional expression is evaluated by Spark rather than by an explicit Python loop.

27 Missing Values

Missing data is common in practical datasets.

Spark DataFrames provide operations through:

```
1 df.na
```

to handle missing values.

Rows containing missing values can be removed:

```
1 clean = df.na.drop()
```

The operation can be restricted to selected columns:

```
1 clean = df.na.drop(  
2     subset=[  
3         "customer_id",  
4         "sales"  
5     ]  
6 )
```

Missing values can also be replaced:

```
1 filled = df.na.fill({  
2     "sales": 0.0,  
3     "city": "unknown"  
4 })
```

Missingness can be inspected using column expressions:

```
1 df.select(  
2     F.col("sales").isNull()  
3 )
```

or:

```
1 df.filter(  
2     F.col("sales").isNull()  
3 )
```

As with pandas, missing-value handling should reflect the statistical and business meaning of the data rather than being treated only as a technical cleaning operation.

28 Distinct Values and Duplicates

Distinct rows can be obtained using:

```
1 df.distinct()
```

Duplicate observations can be removed with:

```
1 df.dropDuplicates()
```

The duplicate criterion can also be restricted to selected columns:

```
1 df.dropDuplicates([  
2     "customer_id"  
3 ])
```

This can be useful when a dataset is expected to contain one observation per business key.

However, removing duplicates should be based on a clearly defined data model. Two rows that happen to contain identical values are not necessarily erroneous duplicates.

29 Sorting

Rows can be sorted using:

```
1 orderBy()
```

For example:

```
1 df.orderBy(  
2     F.col("sales")  
3 )
```

Descending order can be requested with:

```
1 df.orderBy(  
2     F.col("sales").desc()  
3 )
```

Multiple sorting criteria can be combined:

```
1 df.orderBy(  
2   F.col("city"),  
3   F.col("sales").desc()  
4 )
```

Sorting deserves particular attention in distributed computing.

A global ordering may require data to be redistributed across partitions so that the final ordering can be established.

Consequently, sorting a very large distributed dataset can be substantially more expensive than the simplicity of:

```
1 orderBy()
```

might suggest.

This illustrates an important Spark principle:

the syntactic complexity of an operation does not necessarily reflect its distributed computational cost.

30 Aggregation

Aggregations summarise many observations into one or more values.

For example:

```
1 df.select(  
2   F.sum("sales")  
3 )
```

computes total sales.

Several aggregate expressions can be evaluated together:

```

1 df.select(
2     F.count("*").alias(
3         "n"
4     ),
5     F.mean("sales").alias(
6         "mean_sales"
7     ),
8     F.min("sales").alias(
9         "min_sales"
10    ),
11    F.max("sales").alias(
12        "max_sales"
13    )
14 )

```

Common aggregate functions include:

```

1 F.count(...)
2 F.sum(...)
3 F.mean(...)
4 F.avg(...)
5 F.min(...)
6 F.max(...)
7 F.stddev(...)
8 F.variance(...)

```

These operations can require partial results to be computed across partitions and subsequently combined.

For example, a distributed sum can be understood conceptually as:

$$S_j = \sum_{x_i \in D_j} x_i$$

for partition D_j , followed by:

$$S = \sum_{j=1}^p S_j.$$

This ability to compute local partial results and combine them efficiently is particularly suitable for distributed processing.

31 Grouped Aggregations

One of the most common analytical operations is:

```
1  groupBy()
```

For example:

```
1  result = (  
2    df  
3    .groupBy(  
4      "city"  
5    )  
6    .agg(  
7      F.sum("sales").alias(  
8        "total_sales"  
9      ),  
10     F.mean("sales").alias(  
11       "mean_sales"  
12     ),  
13     F.count("*").alias(  
14       "n"  
15     )  
16   )  
17 )
```

Conceptually, the operation partitions observations according to a key:

$$D \longrightarrow \{D_{k_1}, D_{k_2}, \dots, D_{k_m}\},$$

and then computes an aggregation for each group.

For example:

$$S_k = \sum_{i: g_i=k} x_i.$$

The syntax resembles familiar SQL and pandas operations, but the distributed implementation may require observations with the same key to be brought together.

This introduces one of the most important concepts in Spark performance:

data movement.

32 Local Computation and Data Movement

Not all Spark transformations have the same computational cost.

Suppose a partition contains:

```
1 Partition 1:
2 Auckland    100
3 Wellington   80
4
5 Partition 2:
6 Auckland    120
7 Christchurch  90
```

If the task is simply:

```
1 df.withColumn(
2   "sales_tax",
3   F.col("sales") * 0.15
4 )
```

each row can be transformed independently within its existing partition.

There is generally no need for all Auckland observations to be placed together.

However, consider:

```
1 df.groupBy(
2   "city"
3 ).sum(
4   "sales"
5 )
```

To obtain the total for Auckland, Spark must logically combine Auckland observations even when they originate in different partitions.

Data may therefore need to move between executors.

This redistribution of data is called a **shuffle**.

Conceptually:

1	Before shuffle	
2		
3	Partition 1	Partition 2
4	-----	-----
5	Auckland 100	Auckland 120
6	Wellington 80	Christchurch 90
7		
8		
9	↓ redistribution	
10		
11		
12	After shuffle	
13		
14	Partition A	Partition B
15	-----	-----
16	Auckland 100	Wellington 80
17	Auckland 120	Christchurch 90

Network communication is substantially more expensive than many local CPU operations.

For this reason, understanding when an operation causes a shuffle is fundamental to Spark performance.

33 Narrow and Wide Transformations

Spark transformations can be understood broadly in terms of the dependencies between output and input partitions.

A **narrow transformation** is one in which an output partition depends on a limited number of input partitions, often only one.

Examples commonly include:

```

1 select()
2 filter()
3 withColumn()

```

For instance:

$$D_j \longrightarrow T(D_j)$$

can often be computed independently for each partition.

A **wide transformation** requires data from multiple input partitions to contribute to output partitions.

Examples commonly include operations such as:

```
1  groupBy()  
2  orderBy()  
3  distinct()
```

and many joins.

Conceptually:

$$\{D_1, D_2, \dots, D_p\} \longrightarrow \{D'_1, D'_2, \dots, D'_q\}$$

where observations may need to be redistributed according to keys or ordering requirements.

Wide transformations frequently introduce a **shuffle boundary**.

This distinction is more useful for Spark performance reasoning than simply counting how many lines of code an operation requires.

34 Stages

Spark organises execution into **stages**.

A sequence of narrow transformations can often be executed within the same stage because partitions can flow through the transformations without requiring global redistribution.

A shuffle generally creates a boundary between stages.

A simplified plan might look like:

```
1  Read data  
2  |  
3  Filter  
4  |  
5  withColumn  
6  |  
7  |   Stage 1  
8  |
```

```

9  groupBy
10 |
11 SHUFFLE
12 |
13 Aggregation
14 |
15 |   Stage 2
16 |
17 Result

```

Thus, a Spark job can be understood at several levels:

application $\longrightarrow$ jobs $\longrightarrow$ stages $\longrightarrow$ tasks.

Tasks operate on partitions, while stages organise groups of tasks according to execution dependencies.

35 Grouping and Shuffle Cost

Consider:

```

1 result = (
2   df
3   .groupBy(
4     "customer_id"
5   )
6   .agg(
7     F.sum("sales")
8   )
9 )

```

If records for the same `customer_id` are spread across many partitions, Spark must redistribute data so that the required aggregation can be completed.

The computational cost therefore includes more than the arithmetic operation:

$$\sum_i x_i.$$

It may also include:

- serialising records;
- transferring data through the network;
- writing intermediate data;
- reading intermediate data;
- coordinating tasks across executors.

For large workloads, these costs can dominate the arithmetic itself.

This leads to a useful Spark performance principle:

minimise unnecessary movement of data across the cluster.

36 Working with Multiple Aggregations

Spark allows several summaries to be calculated in the same grouped operation:

```
1 summary = (  
2     df  
3     .groupBy(  
4         "city"  
5     )  
6     .agg(  
7         F.count("*").alias(  
8             "n_transactions"  
9         ),  
10        F.sum("sales").alias(  
11            "total_sales"  
12        ),  
13        F.mean("sales").alias(  
14            "mean_sales"  
15        ),  
16        F.max("sales").alias(  
17            "maximum_sale"  
18        )  
19    )  
20 )
```

This is generally preferable to constructing several independent grouped computations when they can naturally be obtained from the same grouping operation.

The code also expresses the analytical intent clearly:

$$\text{city} \longrightarrow \{n, \sum x_i, \bar{x}, \max(x_i)\}.$$

37 Counting Distinct Values

The number of distinct values can be calculated using:

```

1 df.select(
2     F.countDistinct(
3         "customer_id"
4     ).alias(
5         "n_customers"
6     )
7 )

```

For grouped data:

```

1 df.groupBy(
2     "city"
3 ).agg(
4     F.countDistinct(
5         "customer_id"
6     ).alias(
7         "n_customers"
8     )
9 )

```

Distinct counting can be more computationally demanding than ordinary counting because Spark must determine whether values occurring across different partitions refer to the same key.

For very large datasets where an exact answer is unnecessary, Spark also provides approximate alternatives such as:

```

1 F.approx_count_distinct(...)

```

This illustrates another recurring theme in large-scale computing: sometimes a controlled approximation can provide substantial computational savings when exact precision is not required by the problem.

38 Basic Descriptive Statistics

Spark DataFrames provide:

```
1 df.describe()
```

for basic descriptive statistics.

For example:

```
1 df.select(  
2     "sales"  
3 ).describe().show()
```

Another option is:

```
1 df.summary()
```

which can provide additional summary statistics.

These methods are convenient for initial inspection, but for analytical pipelines it is often preferable to specify the required statistics explicitly using:

```
1 agg()
```

because doing so makes the computation and output schema clearer.

39 Collecting Data to the Driver

Spark results can be transferred to the driver using operations such as:

```
1 df.collect()
```

For example:

```
1 rows = df.collect()
```

returns the complete result to the driver as a Python collection.

This is safe only when the result is known to be small.

If a distributed DataFrame contains hundreds of millions of observations, calling:

```
1 collect()
```

attempts to move all of those observations to the driver.

The cluster may have enough aggregate memory to process the dataset while the driver does not have enough memory to contain the complete result.

Thus:

a dataset being processable by Spark does not imply that it can safely be collected into the driver.

For inspection, operations such as:

```
1 df.show()
```

or:

```
1 df.take(10)
```

are usually more appropriate.

For final aggregated results that are demonstrably small:

```
1 collect()
```

may be perfectly reasonable.

The important issue is understanding where the data will reside after the action.

40 Converting to pandas

A Spark DataFrame can be converted to a pandas DataFrame:

```
1 pandas_df = df.toPandas()
```

This transfers the resulting data to the driver.

Therefore, the same fundamental constraint applies:

distributed Spark DataFrame $\xrightarrow{\text{toPandas()}}$ single-machine pandas DataFrame.

The operation should only be used when the resulting dataset is sufficiently small for local memory.

A common workflow is therefore:

```
1 large distributed dataset
2   |
3   v
4 Spark transformations
5   |
6   v
7 Spark aggregations
8   |
9   v
10 small analytical result
11  |
12  v
13 pandas / plotting / local analysis
```

This can be a useful boundary between distributed data preparation and local exploratory or presentation-oriented analysis.

41 DataFrame API as a Declarative Interface

The examples in this section share an important characteristic.

Consider:

```

1 result = (
2     df
3     .filter(
4         F.col("sales") > 0
5     )
6     .withColumn(
7         "revenue",
8         F.col("quantity")
9         * F.col("unit_price")
10    )
11    .groupBy(
12        "city"
13    )
14    .agg(
15        F.sum("revenue").alias(
16            "total_revenue"
17        )
18    )
19 )

```

The code describes the desired transformation at a relatively high level.

It does not explicitly specify:

- which executor handles each row;
- which CPU core performs each multiplication;
- how intermediate records are serialised;
- which network connection transfers grouped records;
- how tasks are scheduled;
- how failed tasks are retried.

Those concerns belong primarily to Spark's execution engine.

The programmer instead expresses a logical computation:

$$D \xrightarrow{\text{filter}} D_1 \xrightarrow{\text{derive revenue}} D_2 \xrightarrow{\text{group}} D_3 \xrightarrow{\text{aggregate}} R.$$

Effective PySpark programming therefore requires reasoning at two levels simultaneously:

1. **analytical logic** — whether the transformation produces the required result;
2. **distributed execution** — whether the logical transformation can be executed efficiently at scale.

The first determines correctness.

The second often determines whether the same correct computation takes seconds, minutes, or hours.

42 Joining DataFrames

Real analytical datasets rarely exist as one complete table.

Information is often distributed across several datasets representing different entities.

For example:

```
1 customers
2   one row per customer
3
4 orders
5   one row per order
```

The two datasets may share:

```
1 customer_id
```

and can therefore be combined.

Consider:

```
1 customers = spark.createDataFrame(
2   [
3     (1, "Alpha"),
4     (2, "Beta"),
5     (3, "Gamma")
6   ],
7   [
8     "customer_id",
9     "customer_name"
10  ]
11 )
12
13 orders = spark.createDataFrame(
14   [
```

```

15     (101, 1, 120.0),
16     (102, 2, 80.0),
17     (103, 1, 95.0),
18     (104, 3, 140.0)
19 ],
20 [
21     "order_id",
22     "customer_id",
23     "amount"
24 ]
25 )

```

The DataFrames can be joined using:

```

1 customer_orders = orders.join(
2     customers,
3     on="customer_id",
4     how="inner"
5 )

```

The resulting DataFrame contains information from both sources.

Its grain remains:

```

1 one row per order

```

because each order corresponds to one customer, while one customer may correspond to several orders.

This relationship is conceptually:

Customers $\longrightarrow$ Orders

with one-to-many cardinality.

As with pandas, understanding the **grain** and **cardinality** of the input datasets is essential before interpreting the result of a join.

43 Join Types

PySpark supports several join types.

Common examples include:

```
1 inner
2 left
3 right
4 full
5 left_semi
6 left_anti
7 cross
```

The appropriate join depends on which population should be preserved.

43.1 Inner Join

An inner join retains observations whose join keys occur on both sides.

```
1 result = orders.join(
2     customers,
3     on="customer_id",
4     how="inner"
5 )
```

If:

$$K_L$$

is the set of keys on the left and:

$$K_R$$

is the set on the right, the retained keys correspond conceptually to:

$$K_L \cap K_R.$$

43.2 Left Join

A left join preserves all rows from the left DataFrame:

```
1 result = orders.join(  
2     customers,  
3     on="customer_id",  
4     how="left"  
5 )
```

If no matching customer exists, columns supplied by the customer DataFrame contain null values.

43.3 Full Join

A full join retains observations appearing on either side:

```
1 result = orders.join(  
2     customers,  
3     on="customer_id",  
4     how="full"  
5 )
```

Conceptually, the key population corresponds to:

$$K_L \cup K_R.$$

44 Semi and Anti Joins

Two particularly useful join types in Spark are:

```
1 left_semi  
2 left_anti
```

A **left semi join** returns rows from the left DataFrame for which a matching key exists in the right DataFrame.

For example:

```
1 customers_with_orders = (  
2     customers  
3     .join(  
4         orders,  
5         on="customer_id",  
6         how="left_semi"  
7     )  
8 )
```

The result contains only columns from:

```
1 customers
```

It answers the question:

Which customers have at least one matching order?

A **left anti join** does the opposite:

```
1 customers_without_orders = (  
2     customers  
3     .join(  
4         orders,  
5         on="customer_id",  
6         how="left_anti"  
7     )  
8 )
```

It answers:

Which customers have no matching order?

These join types are often preferable to constructing a larger join and subsequently filtering it when the objective is fundamentally an existence test.

45 Joining with Explicit Conditions

When the join columns have different names, an explicit Boolean expression can be used.

For example:

```

1 result = (
2     orders
3     .alias("o")
4     .join(
5         customers.alias("c"),
6         F.col("o.customer_id")
7         == F.col("c.id"),
8         how="left"
9     )
10 )

```

Aliases become useful when both DataFrames contain columns with similar or identical names.

For example:

```

1 result.select(
2     F.col("o.order_id"),
3     F.col("c.customer_name")
4 )

```

makes the origin of each column explicit.

46 Joins and Row Multiplication

A join can increase the number of rows substantially.

Suppose key A appears twice in the left DataFrame:

```

1 A
2 A

```

and three times in the right DataFrame:

```

1 A
2 A
3 A

```

The join produces:

$$2 \times 3 = 6$$

matching combinations.

This behaviour is mathematically consistent with join semantics.

The important question is whether the relationship was intended.

In large-scale processing, an accidental many-to-many join can be especially costly because it can create:

- unexpectedly large intermediate datasets;
- additional network traffic;
- increased memory pressure;
- substantially longer execution times.

Therefore, before joining large datasets, it is useful to understand:

1. what one row represents in each dataset;
2. whether the join key is unique on either side;
3. the expected relationship cardinality;
4. the expected grain after the join.

47 Why Joins Can Be Expensive

Suppose two large DataFrames are partitioned independently:

1 Orders partitions:

2
3 P1 P2
4 customer 1 customer 3
5 customer 4 customer 1
6 customer 2 customer 5
7
8

9 Customers partitions:

10
11 P3 P4
12 customer 5 customer 1

```

13 customer 2    customer 3
14         customer 4

```

To join records by:

```

1 customer_id

```

Spark may need records having the same key to become available together.

This can require redistribution across the cluster:

```

1         customer_id
2
3 P1  _____
4 P2  _____> redistribution
5 P3  _____
6 P4  _____

```

This redistribution is a **shuffle**.

The join operation therefore may involve much more than comparing keys.

Its cost can include:

- partitioning records;
- serialisation;
- network transfer;
- writing and reading shuffle data;
- task coordination;
- memory consumption.

For large datasets, the movement of data can dominate the actual key comparison.

48 Shuffle Joins

When both sides of a join are large, Spark may need to redistribute data according to the join key.

Conceptually:

$$D_L \xrightarrow{\text{partition by key}} D'_L$$

and:

$$D_R \xrightarrow{\text{partition by key}} D'_R.$$

Records sharing a key are then brought into compatible partitions so that matching can occur.

This strategy enables distributed joins of very large datasets, but the shuffle can be expensive.

A useful performance question is therefore:

Can this join be completed without redistributing both large datasets?

When one side is sufficiently small, the answer may be yes.

49 Broadcast Joins

Suppose a large transaction dataset must be joined with a relatively small lookup table.

For example:

```
1 transactions
2   hundreds of millions of rows
3
4 product_categories
5   hundreds of rows
```

Instead of redistributing both datasets by the join key, the small dataset can be copied to executors.

Conceptually:

```
1      small table
2      |
3      -----
4      |   |   |
5      v   v   v
6      Executor Executor Executor
7
8 large data:
9   local partition processing
```

Each executor can then join its local portion of the large dataset against the small in-memory table.

PySpark allows a DataFrame to be explicitly marked for broadcast:

```
1 from pyspark.sql.functions import broadcast
2
3 result = large_df.join(
4     broadcast(small_df),
5     on="product_id",
6     how="left"
7 )
```

The key idea is:

move small table to workers

instead of:

shuffle both large and small tables by key.

When appropriate, this can greatly reduce data movement.

50 When Broadcasting Helps

Broadcasting is particularly attractive when one side of the join is genuinely small relative to available executor memory.

Examples may include:

- product category tables;
- country mappings;
- small reference datasets;
- configuration tables;
- compact dimension tables.

However, broadcast is not free.

The broadcasted dataset must be:

- transferred to executors;

- stored in executor memory;
- available to each task that needs it.

Attempting to broadcast a dataset that is too large can create memory pressure and degrade performance.

Thus:

broadcast the small side of a join when its size makes replication across executors economical.

Spark can also choose broadcast strategies automatically when its optimiser has sufficient information, so explicit broadcasting should be based on understanding rather than applied mechanically.

51 Join Strategy versus Join Semantics

It is useful to distinguish two different questions.

The first concerns **logical semantics**:

```
1 inner?  
2 left?  
3 right?  
4 anti?  
5 semi?
```

This determines which rows should appear in the result.

The second concerns **physical strategy**:

```
1 broadcast?  
2 shuffle?  
3 sort?  
4 hash?
```

This determines how Spark executes the logical join.

These are different levels of reasoning:

logical join $\neq$ physical join strategy.

The DataFrame code primarily expresses the logical relationship.

Spark's optimiser selects an execution strategy, although hints and configuration can influence that decision.

52 Partitioning

Spark parallelism is strongly connected to partitions.

The current number of partitions can be inspected through the underlying RDD:

```
1 df.rdd.getNumPartitions()
```

For example:

```
1 n_partitions = (  
2     df  
3     .rdd  
4     .getNumPartitions()  
5 )  
6  
7 n_partitions
```

A good partitioning strategy attempts to balance several competing considerations.

Partitions should generally be:

- numerous enough to provide parallel work;
- large enough to avoid excessive scheduling overhead;
- small enough to fit comfortably within executor resources;
- reasonably balanced in computational load.

There is no universally correct partition count.

The appropriate number depends on:

- dataset size;
- cluster resources;
- workload structure;
- downstream transformations.

53 Repartitioning

A DataFrame can be repartitioned using:

```
1 repartition()
```

For example:

```
1 df_repartitioned = (  
2     df  
3     .repartition(8)  
4 )
```

Spark redistributes the data to construct the requested partitioning.

A column can also be supplied:

```
1 df_repartitioned = (  
2     df  
3     .repartition(  
4         8,  
5         "customer_id"  
6     )  
7 )
```

The resulting partitioning is related to the specified expression.

This can be useful when later computations operate repeatedly on the same key.

However, repartitioning itself requires data movement.

Therefore:

repartitioning should have a reason.

Adding `repartition()` arbitrarily may simply introduce an additional shuffle without reducing downstream cost.

54 Increasing and Decreasing Partitions

A common distinction is:

```
1 repartition()
```

versus:

```
1 coalesce()
```

`repartition()` can be used to increase or decrease partition counts and normally redistributes data.

For example:

```
1 df = df.repartition(100)
```

`coalesce()` is commonly used to reduce the number of partitions while avoiding a full redistribution where possible:

```
1 df = df.coalesce(20)
```

A useful rule of thumb is:

```
1 repartition()  
2     redistribute more fully  
3     useful for rebalancing or increasing partitions  
4  
5 coalesce()  
6     commonly reduce partitions  
7     avoid a full shuffle where possible
```

However, reducing partitions too aggressively can reduce parallelism.

For example:

```
1 df.coalesce(1)
```

may force subsequent work through very limited parallelism.

The smallest possible number of output files is not necessarily the most efficient computational structure.

55 Partition Balance

Suppose a dataset is divided as:

```
1 Partition 1  1 GB
2 Partition 2  1 GB
3 Partition 3  1 GB
4 Partition 4 20 GB
```

The fourth partition will take substantially longer to process.

Even if the first three tasks finish quickly, the stage cannot complete until the slow partition finishes.

This illustrates **data skew**.

Ideal parallel processing would distribute work roughly evenly:

$$|D_1| \approx |D_2| \approx \dots \approx |D_p|.$$

Real data rarely behaves perfectly this way.

Highly uneven key frequencies can therefore create uneven partition sizes and execution times.

56 Data Skew

Consider customer transactions.

Suppose most customers have:

```
1 10–100 transactions
```

while one institutional customer has:

```
1 50 million transactions.
```

A grouping or join on:

```
1 customer_id
```

may place an unusually large amount of work associated with that key into one part of the computation.

Symptoms of skew can include:

- one or a few tasks taking much longer than others;
- large differences in task input size;
- executor memory pressure;
- unexpectedly long shuffle stages.

Skew is important because adding more executors does not necessarily solve a problem whose bottleneck is concentrated in a small number of oversized partitions.

57 More Workers Do Not Guarantee Linear Speedup

Suppose a job takes time:

$$T.$$

Doubling the number of available workers does not imply:

$$T_{\text{new}} = \frac{T}{2}.$$

Distributed workloads contain components that do not scale perfectly.

For example:

- scheduling overhead;
- serialisation;
- network communication;
- shuffle operations;
- skew;
- sequential stages;
- driver-side work.

A more realistic conceptual model is:

$$T = T_{\text{parallel}} + T_{\text{communication}} + T_{\text{coordination}} + T_{\text{sequential}}.$$

Only part of the workload may benefit directly from additional parallel resources.

This is why Spark performance cannot be reduced to:

increase the number of executors.

The execution structure matters.

58 Repeated Computation

Because Spark uses lazy evaluation, the same logical DataFrame can be recomputed when several actions depend on it.

Consider:

```
1 filtered = (  
2   df  
3   .filter(  
4     F.col("sales") > 0  
5   )  
6   .groupBy(  
7     "customer_id"  
8   )  
9   .agg(  
10    F.sum("sales").alias(  
11      "total_sales"  
12    )  
13  )  
14 )
```

Now suppose:

```
1 filtered.count()
```

is executed, followed by:

```
1 filtered.show()
```

Without persistence, Spark may need to execute the required upstream transformations again for the second action.

If the preceding transformations are inexpensive, this may be acceptable.

If they involve:

- expensive joins;
- large scans;
- heavy aggregations;
- repeated iterative use,

recomputation may be undesirable.

59 Caching

Spark can persist a DataFrame using:

```
1 cache()
```

For example:

```
1 customer_summary = (  
2     customer_summary  
3     .cache()  
4 )
```

The first action materialises the required computation:

```
1 customer_summary.count()
```

Subsequent actions can reuse the cached representation rather than necessarily recomputing the complete upstream lineage.

For example:

```
1 customer_summary.show()
```

and:

```
1 customer_summary.count()
```

can then reuse the persisted data.

Caching is therefore conceptually:

expensive computation $\longrightarrow$ materialised reusable result.

60 Persistence

`persist()` provides more explicit control over storage behaviour.

For example:

```
1 from pyspark import StorageLevel
2
3 df.persist(
4     StorageLevel.MEMORY_AND_DISK
5 )
```

Different storage levels determine how persisted partitions can be retained.

The exact choice depends on:

- available executor memory;
- recomputation cost;
- dataset size;
- expected reuse.

`cache()` can be understood as a convenient persistence operation using Spark's default DataFrame caching storage level.

61 When to Cache

Caching is useful when the same expensive DataFrame is reused repeatedly.

Common examples include:

- iterative analysis;
- several downstream actions;
- repeated queries against one transformed dataset;
- some iterative machine learning workflows.

However, caching everything is not a performance strategy.

Cached datasets consume resources.

If a DataFrame is:

- used only once;
- inexpensive to recompute;
- very large;

- competing with other important cached data,

persistence may provide little benefit or even reduce performance.

A useful question is:

Is the cost of storing and maintaining this intermediate result lower than the cost of recomputing it?

62 Removing Cached Data

Persisted data should no longer occupy cache resources once it is unnecessary.

A DataFrame can be removed from persistence using:

```
1 df.unpersist()
```

For example:

```
1 customer_summary.unpersist()
```

Explicitly releasing persisted data can be useful in long-running workflows where several large intermediate datasets are used sequentially.

63 Lineage

Spark keeps information about how a DataFrame was derived.

Conceptually:

```
1 source
2 |
3 filter
4 |
5 select
6 |
7 join
8 |
9 groupBy
10 |
11 aggregate
```

This sequence represents the **lineage** of the computation.

Lineage allows Spark to understand how results can be recomputed if necessary.

It is also closely connected to lazy evaluation because Spark maintains the transformation relationships required to produce a result.

A DataFrame should therefore not be thought of simply as stored rows.

It also carries the logical history required to produce those rows.

64 Reading an Execution Plan

One of the most useful tools for reasoning about Spark execution is:

```
1 explain()
```

For example:

```
1 result.explain()
```

displays the physical plan.

A more detailed representation can be requested using:

```
1 result.explain(  
2     mode="extended"  
3 )
```

or:

```
1 result.explain(  
2     mode="formatted"  
3 )
```

The output can reveal operations such as:

```
1 Scan
2 Filter
3 Project
4 Exchange
5 HashAggregate
6 Sort
7 Join
8 Broadcast
```

The exact plan depends on the workload and Spark's optimiser.

The purpose is not initially to memorise every physical operator.

The first objective is to learn to recognise structural signals.

65 Recognising Exchange

An execution plan containing an operation such as:

```
1 Exchange
```

often indicates that Spark is redistributing data between partitions.

This is therefore an important clue that a shuffle is occurring.

For example, a grouped aggregation may produce a plan conceptually containing:

```
1 Scan
2 |
3 Partial Aggregate
4 |
5 Exchange
6 |
7 Final Aggregate
```

The Exchange separates local partial computation from the redistribution required to complete the global grouping.

66 Partial Aggregation

Distributed aggregations can often reduce communication by performing part of the work locally first.

Suppose a partition contains sales:

100, 120, 80.

Instead of transferring every record immediately, Spark may compute a local partial sum:

$$S_1 = 300.$$

Another partition might compute:

$$S_2 = 450.$$

The final result is then:

$$S = S_1 + S_2 = 750.$$

This pattern reduces the quantity of information that must cross the network.

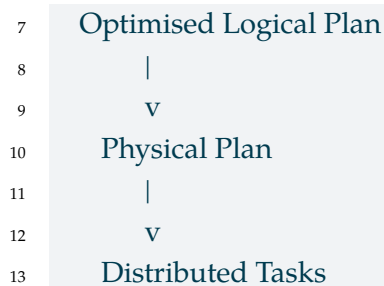
It illustrates why aggregate functions such as sums and counts are particularly suitable for distributed execution.

67 Logical and Physical Plans

Spark separates the logical description of the computation from the physical mechanism used to execute it.

Conceptually:

```
1 PySpark DataFrame operations
2   |
3   v
4 Logical Plan
5   |
6   v
```



For example, the logical request might simply be:

```
1 orders.join(
2   customers,
3   "customer_id"
4 )
```

The physical execution could involve one of several possible join strategies.

This separation allows Spark to optimise the computation without requiring application code to specify every low-level execution decision.

68 Query Optimisation

Spark SQL and DataFrame operations are analysed and optimised before physical execution.

The optimiser can transform logical expressions while preserving their meaning.

Examples of useful optimisations can include:

- eliminating unnecessary columns;
- moving filters closer to data sources;
- simplifying expressions;
- selecting appropriate physical strategies.

The important conceptual point is:

DataFrame code is declarative enough for Spark to reason about the complete query rather than blindly executing each Python expression one at a time.

This is one major reason to prefer built-in Spark expressions over opaque Python logic where possible.

69 Predicate Pushdown

Suppose a large data source contains:

```
1 customer_id
2 country
3 sales
4 date
5 description
6 ...
```

but the query requires only New Zealand records:

```
1 df.filter(
2     F.col("country")
3     == "New Zealand"
4 )
```

For compatible data sources and predicates, Spark may be able to push the filtering operation closer to the data source.

Conceptually:

```
1 less desirable
2
3 Read all data
4 |
5 transfer/process
6 |
7 filter
8
9
10 preferable when supported
11
12 Data source
13 |
14 filter early
15 |
16 read required subset
```

Reducing the amount of data entering later stages can reduce:

- I/O;
- memory pressure;
- network transfer;
- downstream computation.

70 Column Pruning

A similar principle applies to columns.

Suppose a source contains 100 columns, but the analytical computation requires only:

```
1 customer_id
2 sales
```

A query such as:

```
1 df.select(
2     "customer_id",
3     "sales"
4 )
```

allows the optimiser and compatible data source to avoid carrying unnecessary columns through the entire computation.

Reducing data width can lower:

- disk I/O;
- memory use;
- serialisation cost;
- network traffic.

This leads to a useful practical principle:

filter rows and select required columns as early as is logically appropriate.

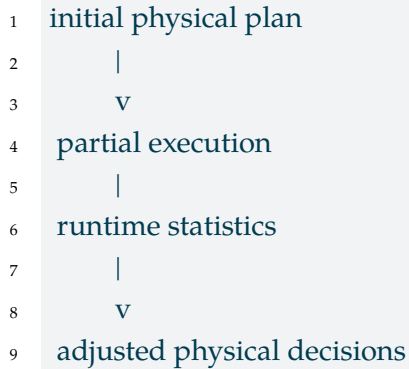
71 Adaptive Query Execution

Some properties of a workload are difficult to know accurately before execution.

Spark can use runtime information to adapt parts of the physical plan while a query is executing.

This general mechanism is known as **Adaptive Query Execution**, or **AQE**.

Conceptually:



The central idea is that optimisation need not rely exclusively on information available before the job begins.

Runtime observations can sometimes support better execution decisions.

For introductory reasoning, the important lesson is not the complete list of AQE features.

It is this:

the physical plan can be influenced by information discovered during execution.

This reinforces the distinction between the logical DataFrame program and its physical implementation.

72 Data Skew and Adaptive Execution

One situation in which runtime information can be useful is skew.

Suppose most shuffle partitions are moderate:

```
1 100 MB
2 110 MB
3 95 MB
```

while one partition is:

```
1 8 GB.
```

That partition may become a severe straggler.

Modern Spark execution can use runtime information in some situations to mitigate skewed shuffle partitions.

The broader principle remains:

balanced partition sizes matter because a stage completes only when its slowest required tasks complete.

73 Built-In Functions versus Python UDFs

Spark provides a large collection of built-in expressions through:

```
1 pyspark.sql.functions
```

For example:

```
1 F.upper(...)
2 F.when(...)
3 F.sum(...)
4 F.substring(...)
5 F.regexp_replace(...)
```

When equivalent built-in functionality exists, it is generally preferable to express the transformation using Spark's native expressions.

For example:

```
1 df.withColumn(
2     "city_upper",
3     F.upper(
4         F.col("city")
5     )
6 )
```

is preferable to introducing custom Python logic merely to convert a string to uppercase.

Built-in expressions remain visible to Spark's query engine and integrate naturally with optimization and execution planning.

74 User-Defined Functions

Sometimes the required logic is not available through existing Spark expressions.

PySpark allows custom functions to be defined as UDFs.

Conceptually:

```
1 from pyspark.sql.types import DoubleType
2
3 @F.udf(
4     returnType=DoubleType()
5 )
6 def custom_function(x):
7     ...
```

The function can then participate in a DataFrame expression.

However, custom UDFs should not be the first solution simply because Python makes them easy to write.

They introduce an additional abstraction boundary between Spark's execution engine and custom Python logic.

A useful preference order is:

```
1 built-in Spark expression
2     ↓
3 SQL expression
4     ↓
5 specialised vectorised mechanism when appropriate
6     ↓
7 ordinary Python UDF when genuinely necessary
```

The exact choice depends on the operation, but preserving Spark-visible expressions generally gives the execution engine more information to work with.

75 Performance Reasoning

Spark performance tuning is not primarily about memorising configuration parameters.

A more useful sequence of questions is:

1. What is the grain of the data?
2. How large are the datasets?
3. How many partitions exist?
4. Are partitions reasonably balanced?
5. Which operations cause shuffles?
6. Are large datasets being joined unnecessarily?
7. Can a genuinely small join side be broadcast?
8. Is the same expensive computation repeated?
9. Is caching justified?
10. Are too many rows or columns being read?
11. Is data being unnecessarily collected to the driver?
12. What does `explain()` reveal about the execution plan?

These questions connect the analytical code with the physical computation.

76 A Simple Optimisation Example

Consider:

```
1 result = (  
2   transactions  
3   .join(  
4     products,  
5     on="product_id"  
6   )  
7   .filter(  
8     F.col("country")  
9     == "New Zealand"  
10  )  
11  .groupBy(  
12    "category"  
13  )  
14  .agg(  
15    F.sum("amount").alias(  
16      "total_amount"  
17    )  
18  )  
19 )
```

Suppose:

```
1 transactions
2     extremely large
3
4 products
5     very small
```

Several questions should immediately arise.

First:

Can the transaction filter logically be applied before the join?

If so:

```
1 nz_transactions = (
2     transactions
3     .filter(
4         F.col("country")
5         == "New Zealand"
6     )
7 )
```

may reduce the amount of data entering the join.

Second:

Is products sufficiently small to broadcast?

If so:

```
1 result = (
2     nz_transactions
3     .join(
4         F.broadcast(products),
5         on="product_id"
6     )
7     .groupBy(
8         "category"
9     )
10    .agg(
11        F.sum("amount").alias(
12            "total_amount"
```

```
13 )  
14 )  
15 )
```

may avoid a more expensive join strategy.

Third:

Which columns are actually needed?

Selecting only required columns before expensive downstream operations may reduce the amount of data carried through the plan.

The important point is not that these changes are universally correct.

The point is that Spark optimisation follows from reasoning about:

```
1 data volume  
2 partitioning  
3 data movement  
4 reuse  
5 execution plan
```

rather than from arbitrary rewriting of syntax.

77 Spark Performance as Data-Movement Management

Many expensive Spark workloads can ultimately be understood in terms of moving information.

Operations may move data:

```
1 storage → executors  
2 executor → executor  
3 executors → driver  
4 memory → disk  
5 disk → memory
```

Examples include:

```
1 large scan
2   storage → executors
3
4 shuffle
5   executor → executor
6
7 collect()
8   executors → driver
9
10 spill
11   memory → disk
12
13 cache read
14   persisted storage → computation
```

A useful mental model is therefore:

Large-scale data processing is often constrained as much by data movement as by arithmetic.

A multiplication such as:

$$x_i y_i$$

is computationally inexpensive.

Moving terabytes of records through a network so that the appropriate x_i and y_i become colocated can be far more expensive.

This difference explains much of the gap between ordinary local Python reasoning and effective distributed computing.

78 A Practical Mental Model

When reading PySpark code, it is useful to analyse it in two passes.

First, ask about the **logical computation**:

- 1 What data are selected?
- 2 What rows are filtered?
- 3 What variables are created?
- 4 What groups are formed?
- 5 What tables are joined?
- 6 What result is required?

Then ask about the **physical implications**:

- 1 How are the data partitioned?
- 2 Which operations remain local?
- 3 Where are shuffles required?
- 4 Could skew occur?
- 5 Can a small table be broadcast?
- 6 Will an intermediate result be reused?
- 7 Does the result move to the driver?

The first pass establishes correctness.

The second establishes scalability.

A Spark program that returns the correct answer but moves vastly more data than necessary may be logically correct while being operationally poor.

Effective PySpark work therefore requires both forms of reasoning.

79 Spark SQL

Spark DataFrames and Spark SQL operate on the same underlying structured-data engine.

A DataFrame can therefore be queried either through:

- the DataFrame API;
- SQL expressions.

This does not represent two separate execution systems.

Both interfaces ultimately describe structured computations that Spark analyses and executes.

For example, suppose:


```

1 sales = spark.createDataFrame(
2     [
3         (1, "Auckland", 120.0),
4         (2, "Wellington", 95.0),
5         (3, "Auckland", 140.0),
6         (4, "Christchurch", 80.0)
7     ],
8     [
9         "customer_id",
10        "city",
11        "sales"
12    ]
13 )

```

A grouped aggregation can be expressed with the DataFrame API:

```

1 result = (
2     sales
3     .groupBy(
4         "city"
5     )
6     .agg(
7         F.sum("sales").alias(
8             "total_sales"
9         )
10    )
11 )

```

The same analytical operation can be expressed through SQL.

80 Temporary Views

A DataFrame can be exposed as a temporary SQL view:

```

1 sales.createOrReplaceTempView(
2     "sales"
3 )

```

SQL can then be executed using:

```

1 result = spark.sql("""
2     SELECT
3         city,
4         SUM(sales) AS total_sales
5     FROM sales
6     GROUP BY city
7 """)

```

The returned object is again a Spark DataFrame.

Thus:

Spark SQL $\rightarrow$ Spark DataFrame.

The SQL query is not returning a fundamentally different data structure.

It provides another interface for constructing the same kind of distributed logical computation.

81 DataFrame API versus SQL

There is no universal requirement to use exclusively SQL or exclusively the DataFrame API.

Both have strengths.

The DataFrame API integrates naturally with Python code:

```

1 result = (
2     df
3     .filter(
4         F.col("sales") > 0
5     )
6     .groupBy(
7         "city"
8     )
9     .agg(
10        F.sum("sales").alias(
11            "total_sales"
12        )
13    )
14 )

```

SQL can be particularly readable for relational operations:

```
1 SELECT
2   city,
3   SUM(sales) AS total_sales
4 FROM sales
5 WHERE sales > 0
6 GROUP BY city
```

A practical Spark codebase may use both.

The important point is that both are declarative interfaces over Spark's structured query engine.

The choice should therefore depend primarily on:

- readability;
- team conventions;
- maintainability;
- the nature of the transformation.

82 SQL Joins

Temporary views also allow relational operations to be expressed naturally.

Suppose:

```
1 orders.createOrReplaceTempView(
2   "orders"
3 )
4
5 customers.createOrReplaceTempView(
6   "customers"
7 )
```

A join can be expressed as:

```
1 customer_orders = spark.sql("""
2   SELECT
3     o.order_id,
4     o.amount,
```

```

5     c.customer_name
6 FROM orders AS o
7 LEFT JOIN customers AS c
8     ON o.customer_id = c.customer_id
9 """)

```

The logical join semantics are equivalent to an appropriate DataFrame join.

Spark's optimiser can then determine an execution strategy for the query.

83 Window Functions

Aggregations usually change the grain of a dataset.

Suppose a table contains:

```

1 one row per transaction

```

and we compute:

```

1 df.groupBy(
2     "customer_id"
3 ).agg(
4     F.sum("amount")
5 )

```

The grain becomes:

```

1 one row per customer

```

Sometimes, however, a group-level quantity is required **without reducing the original number of rows**.

This is where **window functions** are particularly useful.

A window defines a set of related rows over which a function is evaluated while retaining row-level output.

84 Defining a Window

Window specifications are created using:

```
1 from pyspark.sql.window import Window
```

Suppose transactions should be analysed independently for each customer:

```
1 customer_window = (  
2     Window  
3     .partitionBy(  
4         "customer_id"  
5     )  
6 )
```

A total can then be calculated:

```
1 result = df.withColumn(  
2     "customer_total",  
3     F.sum(  
4         "amount"  
5     ).over(  
6         customer_window  
7     )  
8 )
```

Each transaction remains in the output.

The additional column contains the total associated with that transaction's customer.

The distinction is:

```
1 groupBy() + aggregation  
2     many rows → fewer rows  
3  
4 window function  
5     many rows → same number of rows
```

This is analogous to the distinction between aggregation and group-level transformation considered earlier with pandas.

85 Ordered Windows

A window can also define an ordering.

For example:

```
1 customer_time_window = (  
2     Window  
3     .partitionBy(  
4         "customer_id"  
5     )  
6     .orderBy(  
7         "order_date"  
8     )  
9 )
```

This allows operations that depend on row sequence within each customer.

Examples include:

```
1 F.row_number()  
2 F.rank()  
3 F.dense_rank()  
4 F.lag()  
5 F.lead()
```

86 Row Number

Suppose orders should be numbered chronologically for each customer.

```
1 result = df.withColumnn(  
2     "order_number",  
3     F.row_number().over(  
4         customer_time_window  
5     )  
6 )
```

Conceptually:

```

1 Customer A
2   first order → 1
3   second order → 2
4   third order → 3
5
6 Customer B
7   first order → 1
8   second order → 2

```

The ordering resets for each partition defined by:

```

1 customer_id

```

87 Ranking

Suppose sales should be ranked within each city:

```

1 ranking_window = (
2   Window
3   .partitionBy(
4     "city"
5   )
6   .orderBy(
7     F.col(
8       "sales"
9     ).desc()
10  )
11 )

```

Then:

```

1 result = df.withColumn(
2   "rank",
3   F.rank().over(
4     ranking_window
5   )
6 )

```

`rank()` and `dense_rank()` behave differently when ties occur.

For example:

```
1 values
2 100
3 100
4 90
```

ordinary rank is conceptually:

```
1 1
2 1
3 3
```

while dense rank is:

```
1 1
2 1
3 2
```

The appropriate function depends on the required semantics.

88 Previous and Next Observations

`lag()` retrieves a preceding value within a window.

For example:

```
1 result = df.withColumn(
2     "previous_sales",
3     F.lag(
4         "sales",
5         1
6     ).over(
7         customer_time_window
8     )
9 )
```

A change from the previous observation can then be calculated:


```

1 result = result.withColumn(
2     "sales_change",
3     F.col("sales")
4     - F.col("previous_sales")
5 )

```

Similarly:

```

1 F.lead(...)

```

can access subsequent observations.

These operations are particularly useful for:

- temporal features;
- change detection;
- customer histories;
- event sequences.

89 Running Aggregations

Windows can also define row boundaries.

For example, a cumulative customer total can be calculated using:

```

1 running_window = (
2     Window
3     .partitionBy(
4         "customer_id"
5     )
6     .orderBy(
7         "order_date"
8     )
9     .rowsBetween(
10        Window.unboundedPreceding,
11        Window.currentRow
12    )
13 )

```

Then:

```

1 result = df.withColumn(
2     "cumulative_sales",
3     F.sum(
4         "sales"
5     ).over(
6         running_window
7     )
8 )

```

For each row, the result represents:

$$C_t = \sum_{i=1}^t x_i.$$

The original row grain remains unchanged.

90 Window Functions and Performance

Window functions can be computationally expensive.

Operations such as:

```

1 partitionBy(...)
2 orderBy(...)

```

may require Spark to redistribute and sort data so that the window semantics can be satisfied.

A simple-looking expression such as:

```

1 F.row_number().over(window)

```

may therefore involve:

- shuffle;
- partitioning;
- sorting;
- distributed state management.

As elsewhere in Spark, logical simplicity does not imply physical simplicity.

91 Writing Data

Spark DataFrames can be written to external storage using:

```
1 df.write
```

For example:

```
1 (  
2   df  
3   .write  
4   .mode("overwrite")  
5   .parquet(  
6     "output/sales"  
7   )  
8 )
```

Common write modes include:

```
1 append  
2 overwrite  
3 error  
4 ignore
```

The appropriate mode depends on the data pipeline semantics.

For example:

```
1 .mode("overwrite")
```

should not be used casually in production because it changes existing output data.

92 Writing CSV

CSV output can be written using:

```
1 (
2   df
3   .write
4   .mode("overwrite")
5   .option(
6     "header",
7     True
8   )
9   .csv(
10    "output/sales_csv"
11  )
12 )
```

An important difference from local pandas workflows is that Spark output usually corresponds to a **directory containing multiple part files**.

For example:

```
1 output/sales_csv/
2   part-00000-...
3   part-00001-...
4   part-00002-...
```

This follows naturally from distributed execution: several partitions can be written independently.

93 Why Spark Produces Multiple Files

Suppose a DataFrame contains eight partitions.

Conceptually:

```
1 Partition 1 → file 1
2 Partition 2 → file 2
3 ...
4 Partition 8 → file 8
```

Writing partitions independently allows Spark to preserve parallelism.

Forcing all output through:

```
1 coalesce(1)
```

may create one output partition, but it can also create a bottleneck.

Therefore:

a single output file is not automatically a desirable Spark output architecture.

Distributed storage formats are normally designed to operate efficiently over collections of files.

94 Parquet

Parquet is a columnar storage format widely used with Spark.

Unlike row-oriented text formats such as CSV, Parquet stores data in a structured columnar representation.

Conceptually:

```
1 Row-oriented
```

```
2
```

```
3 row 1:
```

```
4 id, city, sales
```

```
5
```

```
6 row 2:
```

```
7 id, city, sales
```

```
8
```

```
9
```

```
10 Column-oriented
```

```
11
```

```
12 id:
```

```
13 1, 2, ...
```

```
14
```

```
15 city:
```

```
16 Auckland, Wellington, ...
```

```
17
```

```
18 sales:
```

```
19 120, 95, ...
```

Columnar storage is particularly useful for analytical workloads where queries frequently access only a subset of columns.

For example:

```
1 df.select(  
2     "customer_id",  
3     "sales"  
4 )
```

does not necessarily require reading every unrelated column from a suitably organised Parquet dataset.

95 Why Parquet Is Often Preferable to CSV

CSV has several advantages:

- simple;
- portable;
- human-readable;
- widely supported.

However, CSV does not naturally preserve rich schema information and is relatively inefficient for many large analytical workloads.

Parquet can provide benefits such as:

- columnar storage;
- schema preservation;
- compression;
- efficient column pruning;
- compatibility with predicate pushdown;
- efficient distributed reads.

Therefore, a common large-scale pipeline may look like:

```
1 raw source  
2 ↓  
3 Spark ingestion  
4 ↓  
5 cleaning / transformation  
6 ↓
```

```
7 Parquet or table format
8 ↓
9 downstream analytics
```

CSV remains useful for interchange and small exports, but Parquet is generally more natural for large analytical datasets.

96 Partitioned Output

Data can also be physically organised by column values.

For example:

```
1 (
2   df
3   .write
4   .partitionBy(
5     "year",
6     "month"
7   )
8   .parquet(
9     "output/transactions"
10  )
11 )
```

The resulting directory structure may conceptually resemble:

```
1 transactions/
2   year=2025/
3     month=1/
4     month=2/
5   year=2026/
6     month=1/
7     month=2/
```

If later queries require only:

```
1 year = 2026
2 month = 2
```

the storage layout may allow irrelevant partitions to be skipped.

This technique is commonly called **partition pruning** when the query engine avoids reading unrelated physical partitions.

97 Choosing Partition Columns

Physical partitioning should reflect realistic query patterns.

A useful partition column often has:

- meaningful filtering value;
- moderate cardinality;
- reasonably balanced data volumes.

For example, partitioning by:

```
1 year
2 month
```

can be reasonable for time-based datasets.

Partitioning by a nearly unique identifier such as:

```
1 transaction_id
```

could generate enormous numbers of tiny directories or files and would generally be inappropriate.

Thus:

physical partitioning should reflect expected access patterns rather than simply choosing available columns.

98 The Small Files Problem

Distributed systems can produce very large numbers of small files.

For example:

```
1 1,000,000 files
2 each approximately 20 KB
```


may be much less efficient than:

```
1 hundreds of appropriately sized files
```

even if both contain the same total data volume.

Large numbers of tiny files can increase:

- metadata operations;
- task scheduling;
- file-open overhead;
- planning complexity.

This is commonly referred to as the **small files problem**.

Partition counts and output structure therefore affect not only computation but also downstream storage efficiency.

99 Spark and pandas

Spark and pandas both expose DataFrame abstractions, but they target different computational settings.

A pandas workflow is typically:

```
1 single Python process
2   |
3 in-memory DataFrame
4   |
5 local computation
```

A Spark workflow is typically:

```
1 distributed data
2   |
3 logical DataFrame plan
4   |
5 distributed execution
```

The key difference is not merely syntax.

It is the execution model.

100 Moving from Spark to pandas

For sufficiently small results:

```
1 pdf = df.toPandas()
```

can move Spark data into pandas.

This operation changes the computational model:

distributed $\longrightarrow$ single-machine.

Therefore, it should only be used when the resulting dataset fits comfortably in driver memory.

A sensible pattern may be:

```
1 large raw data
2   ↓
3 Spark
4   ↓
5 filter
6   ↓
7 aggregate
8   ↓
9 small result
10  ↓
11 pandas
12  ↓
13 visualisation / local analysis
```

The boundary should occur after the distributed workload has reduced the data sufficiently.

101 Apache Arrow

Moving data between Spark's execution environment and Python objects can involve serialisation and conversion overhead.

Apache Arrow provides a standardised columnar in-memory representation that can improve certain data transfers between Spark and Python ecosystems.

In PySpark, Arrow can be used in operations such as:

```
1 DataFrame.toPandas()
```

and:

```
1 SparkSession.createDataFrame(  
2     pandas_dataframe  
3 )
```

when Arrow optimisation is enabled.

Conceptually:

```
1 Spark columnar data  
2 |  
3 Arrow  
4 |  
5 Python / pandas
```

rather than converting every value through a less efficient object-by-object representation.

Arrow reduces some communication and serialisation overhead.

It does **not** remove the fundamental memory constraint of:

```
1 toPandas()
```

The complete result still ends up in the driver process.

Therefore:

Arrow can make a transfer more efficient without making an arbitrarily large local transfer safe.

102 Pandas UDFs

Spark can also execute vectorised Python logic using **Pandas UDFs**.

A simple example can be written using:

```

1 import pandas as pd
2
3 @F.pandas_udf("double")
4 def centre_values(
5     values: pd.Series
6 ) -> pd.Series:
7
8     return (
9         values
10        - values.mean()
11    )

```

The function can then participate in a Spark expression.

Pandas UDFs use pandas objects within vectorised batches and use Arrow for data exchange.

They can therefore be more efficient than traditional row-by-row Python UDF patterns for suitable workloads. [contentReferenceoaicite:1](#)

However, this still does not imply that every transformation should become a Pandas UDF.

103 Built-In Expressions First

A useful preference remains:

```

1 Spark built-in expression
2   ↓
3 Spark SQL expression
4   ↓
5 specialised vectorised UDF
6   ↓
7 custom Python UDF

```

For example, if the desired transformation is:

```

1 F.log(
2     F.col("sales")
3 )

```

there is little reason to write:

```

1 @F.pandas_udf(...)
2 def log_sales(...):
3     ...

```

The built-in Spark operation is simpler and remains fully visible to Spark’s query engine.

Custom UDFs are most appropriate when genuinely custom logic cannot be represented through available expressions.

104 Machine Learning with Spark

Spark also provides a machine learning library through:

```

1 pyspark.ml

```

The high-level API is DataFrame-based.

It follows an object model built around concepts such as:

```

1 Transformer
2 Estimator
3 Model
4 Pipeline
5 PipelineModel

```

An **Estimator** learns from data.

Conceptually:

$$\text{Estimator} + \text{training data} \xrightarrow{\text{fit()}} \text{Model}.$$

A **Transformer** transforms one DataFrame into another:

$$D \xrightarrow{\text{transform()}} D'.$$

A fitted model is itself generally a Transformer because it transforms input rows by adding predictions or other outputs.

105 Estimator and Transformer

This structure closely resembles the object-oriented design discussed earlier in these notes.

For example:

```
1 Estimator
2     configuration
3     fit()
4
5 Model
6     learned state
7     transform()
```

A feature transformer may be used as:

```
1 transformer.transform(
2     df
3 )
```

while a learning algorithm is used as:

```
1 model = estimator.fit(
2     training_df
3 )
```

and then:

```
1 predictions = model.transform(
2     test_df
3 )
```

This is a distributed analogue of the estimator-style interfaces used across Python machine learning ecosystems.

106 Feature Vectors

Many Spark ML algorithms expect predictors to be represented in a single vector column.

Suppose the input contains:

```
1 age
2 income
3 tenure
```

A VectorAssembler can combine them:

```
1 from pyspark.ml.feature import VectorAssembler
2
3 assembler = VectorAssembler(
4     inputCols=[
5         "age",
6         "income",
7         "tenure"
8     ],
9     outputCol="features"
10 )
```

Then:

```
1 assembled = assembler.transform(
2     df
3 )
```

produces a column:

```
1 features
```

containing the model input vector.

Conceptually:

$$(x_1, x_2, x_3) \longrightarrow \mathbf{x}.$$

107 A Simple Spark ML Estimator

For example:

```

1 from pyspark.ml.regression import (
2     LinearRegression
3 )
4
5 regression = LinearRegression(
6     featuresCol="features",
7     labelCol="target"
8 )

```

The estimator can be fitted:

```

1 model = regression.fit(
2     training_data
3 )

```

and predictions produced:

```

1 predictions = model.transform(
2     test_data
3 )

```

The fitted object contains state learned from the training data.

Again, the lifecycle is:

```

1 configure
2   ↓
3 fit
4   ↓
5 learned model
6   ↓
7 transform / predict

```

108 Spark ML Pipelines

Several transformations and estimators can be combined using:

1 Pipeline

For example:

```
1 from pyspark.ml import Pipeline
2
3 pipeline = Pipeline(
4     stages=[
5         assembler,
6         regression
7     ]
8 )
```

Calling:

```
1 pipeline_model = pipeline.fit(
2     training_data
3 )
```

runs the stages in order.

A fitted estimator stage becomes a fitted model, while transformer stages continue transforming the intermediate DataFrame.

The resulting PipelineModel contains the fitted stages and can be applied to new data. :contentReferenceoaicite:3

This is conceptually:

```
1 raw DataFrame
2   |
3   v
4 feature transformer
5   |
6   v
7 assembled DataFrame
8   |
9   v
10 estimator.fit()
11  |
12  v
13 fitted pipeline
```

109 PySpark and Large-Scale Machine Learning

Spark ML can be useful when:

- training data is already distributed;
- feature engineering requires large-scale Spark transformations;
- the supported algorithms fit the modelling problem;
- integration with an existing Spark data platform is important.

However, Spark should not automatically be chosen for every model because the raw dataset is large.

A common workflow may instead be:

```
1 large distributed raw data
2 |
3 Spark
4 |
5 feature engineering
6 |
7 aggregation / sampling
8 |
9 smaller modelling dataset
10 |
11 scikit-learn / XGBoost / other framework
```

The appropriate architecture depends on the problem.

Spark is often particularly valuable in the **data preparation layer**, even when model training eventually occurs using another system.

110 Spark versus pandas for Machine Learning Preparation

Suppose a training dataset contains:

```
1 5 million rows
2 50 columns
```

and fits comfortably in memory on an appropriate machine.

A pandas-based workflow may still be simpler.

Now suppose the raw source consists of:

```
1 20 TB of transaction history
```

from which a customer-level modelling table must be produced.

Spark may be particularly valuable for:

```
1 20 TB transactions
2   ↓
3  filter
4   ↓
5  join
6   ↓
7  aggregate
8   ↓
9  one row per customer
10  ↓
11 modelling dataset
```

Thus, the need for Spark can originate from **feature construction and data reduction**, even if the final model itself does not require distributed training.

111 Data Leakage in Spark Workflows

Distributed execution does not change statistical principles.

Suppose a model predicts churn at the beginning of each month.

If Spark computes:

```
1 customer_history = (
2   transactions
3   .groupBy(
4     "customer_id"
5   )
6   .agg(
7     F.sum("amount")
8   )
9 )
```

using transactions from the complete future history, then the resulting feature may contain information unavailable at prediction time.

Spark can compute the feature correctly from a computational perspective while the feature is statistically invalid.

The same principle applies to:

- train/test preprocessing;
- target-derived aggregations;
- global statistics;
- temporal features;
- future joins.

Therefore:

distributed correctness and statistical correctness are separate requirements.

112 Common PySpark Mistakes

Spark code can execute successfully while still being poorly designed.

Several recurring mistakes are particularly important.

112.1 Treating Spark Like pandas

Operations that are trivial locally may be distributed jobs in Spark.

For example:

```
1 df.count()
```

may scan a very large dataset.

Similarly:

```
1 df.orderBy(...)
```

may require a global shuffle.

Spark methods should therefore be interpreted through their execution implications rather than only their familiar DataFrame syntax.

112.2 Calling collect() Too Early

A pattern such as:

```
1 rows = df.collect()
```

moves the complete result to the driver.

Doing this before sufficiently reducing a distributed dataset defeats the purpose of distributed processing and may exhaust driver memory.

112.3 Calling toPandas() on Large Data

Similarly:

```
1 df.toPandas()
```

is appropriate only when the result is safely manageable on one machine.

Arrow may improve the transfer mechanism, but it does not change this fundamental constraint.
:contentReferenceoaicite:4

112.4 Excessive Use of Python UDFs

Custom UDFs can obscure operations from Spark's native expression system and add Python execution overhead.

Built-in functions should generally be preferred when equivalent functionality exists.

112.5 Unnecessary Shuffles

Repeated operations such as:

```
1 groupBy()  
2 orderBy()  
3 repartition()  
4 join()  
5 distinct()
```

can cause substantial redistribution.

A workflow containing many unnecessary wide transformations may spend much of its time moving rather than processing data.

112.6 Repartitioning Without a Reason

Adding:

```
1 repartition(200)
```

because a particular number appears in an example is not a performance strategy.

Partition counts should follow:

- data volume;
- cluster resources;
- workload;
- skew;
- downstream structure.

112.7 Caching Everything

Persisting every intermediate DataFrame consumes executor storage and may degrade performance.

Cache only when reuse and recomputation cost justify it.

112.8 Ignoring Skew

A cluster can contain many idle executors while one task processes an extremely large skewed partition.

More hardware does not automatically solve uneven work distribution.

112.9 Producing Too Many Small Files

Over-partitioned writes can create operationally inefficient storage layouts.

The computation may finish successfully while producing poor downstream data organisation.

112.10 Ignoring the Execution Plan

Spark provides:

```
1 explain()
```

precisely because the physical execution may differ substantially from what the surface syntax suggests.

When performance is surprising, inspecting the plan is often more useful than guessing.

113 A Practical Spark Workflow

A useful way to approach a new PySpark task is to proceed in stages.

First, understand the data:

```
1 What does one row represent?  
2 What are the keys?  
3 What are the schemas?  
4 How large are the sources?  
5 Where do they live?
```

Then construct the logical computation:

```
1 select required columns  
2 filter required rows  
3 derive features  
4 join related data  
5 aggregate to required grain
```

Then consider distributed implications:

```
1 Where are shuffles?  
2 Which side of each join is small?  
3 Are partitions balanced?  
4 Is an intermediate reused?  
5 Could skew occur?
```

Then inspect the execution plan:

```

1 result.explain(
2     mode="formatted"
3 )

```

Finally, materialise only what is necessary:

```

1 write distributed output
2 or
3 collect only a small final result

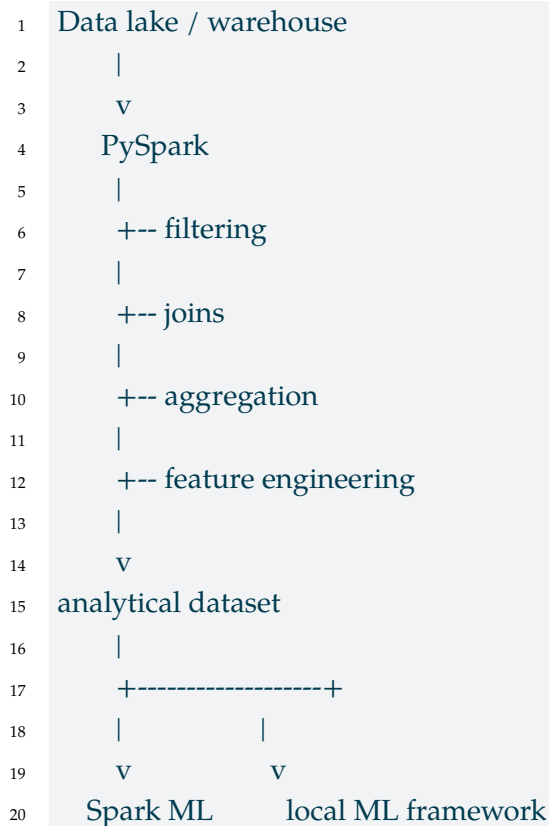
```

This sequence separates logical correctness from performance reasoning without treating them as unrelated concerns.

114 PySpark in a Data Science Workflow

PySpark often occupies the part of the workflow between raw large-scale data and an analytical dataset.

For example:



This positioning helps explain why Spark knowledge is relevant even for data scientists who do not spend most of their time developing distributed infrastructure.

The data required for modelling often has to be constructed before modelling can begin.

115 Spark as an Execution Engine

The most useful way to think about Spark is not simply:

pandas for bigger datasets.

Spark is better understood as a **distributed execution engine for data-processing plans**.

Its DataFrame API happens to expose a familiar tabular abstraction, but the important concepts underneath are:

- partitions;
- tasks;
- stages;
- lazy evaluation;
- execution plans;
- shuffles;
- distributed joins;
- persistence;
- fault-tolerant recomputation;
- query optimisation.

This explains both Spark's power and its complexity.

116 Choosing Spark

A useful decision framework is:

```
1 Does the problem fit comfortably on one machine?
2 |
3 yes
4 |
5 v
6 prefer simpler local tools
7 when they meet the requirement
8
```

```
9
10 Does the workload require distributed scale
11 or already live in distributed infrastructure?
12 |
13 yes
14 |
15 v
16 Spark may be appropriate
```

This avoids two opposite mistakes:

1. attempting to solve genuinely distributed workloads with local tools;
2. introducing a distributed system into a problem that does not require one.

The goal is not to maximise technological complexity.

It is to select the simplest computational model that reliably handles the workload.

117 Key Takeaways

Apache Spark is a distributed computing engine designed to process data across multiple computational resources.

PySpark provides a Python interface to Spark while preserving Spark's distributed execution model.

The need for Spark typically arises when data volume, computational requirements, or system architecture exceed what is conveniently handled by a single machine.

Horizontal scaling distributes work across several machines, but distributed computation introduces communication, scheduling, serialisation, and coordination costs. More machines therefore do not imply proportional speedup.

Spark applications are coordinated by a driver and execute tasks across executors. Data is divided into partitions, which form a central unit of distributed parallelism.

Spark DataFrames are distributed tabular abstractions with explicit schemas. Unlike pandas DataFrames, they represent both structured data and a logical computation to be executed by Spark.

Spark uses lazy evaluation. Transformations build a computation plan, while actions trigger execution.

Operations such as `select()`, `filter()`, and many column transformations can often operate locally within partitions. Operations such as grouped aggregations, global sorting, repartitioning, and many joins may require a shuffle.

A shuffle redistributes data between partitions and executors. Since network and storage movement can dominate arithmetic cost, understanding data movement is fundamental to Spark performance.

Joins should be analysed in terms of both logical semantics and physical execution. Grain, key uniqueness, cardinality, and expected row multiplication remain fundamental analytical concepts.

When one side of a join is sufficiently small, a broadcast strategy can avoid redistributing both sides of the join.

Partitioning controls available parallelism and influences memory use, scheduling overhead, shuffle behaviour, and output layout. `repartition()` and `coalesce()` should therefore be used for explicit computational reasons rather than arbitrary partition-count targets.

Data skew occurs when work is distributed unevenly among partitions. A small number of exceptionally large partitions can dominate total stage execution time.

Caching and persistence can avoid repeated expensive computation, but persisted data consumes cluster resources. Caching is useful when reuse justifies its storage cost.

Execution plans expose how Spark intends to evaluate a `DataFrame` computation. `explain()` is therefore an important tool for connecting high-level PySpark code with physical distributed execution.

Spark's structured query engine can be accessed through both the `DataFrame` API and Spark SQL. The appropriate interface depends primarily on clarity and maintainability because both ultimately use Spark's structured execution engine.

Window functions compute group-relative or ordered quantities while preserving row-level grain. They are useful for ranking, cumulative calculations, previous/next observations, and temporal feature construction.

Columnar formats such as Parquet are particularly well suited to analytical Spark workloads because they support efficient column selection, compression, schema preservation, and optimisation opportunities such as column pruning and predicate pushdown.

Spark writes distributed results naturally as multiple files. The number and organisation of output files should be treated as part of data-system design rather than forcing local single-file conventions onto distributed workflows.

Apache Arrow can improve certain Spark–Python data transfers and enables vectorised mechanisms such as Pandas UDFs, but it does not remove the memory limitations of collecting distributed data onto the driver. :contentReferenceoaicite:5

Spark ML follows an estimator/transformer architecture. Estimators learn models through `fit()`, transformers modify DataFrames through `transform()`, and pipelines combine multiple stages into reusable modelling workflows. :contentReferenceoaicite:6

Spark is often valuable in machine learning not only for distributed model training, but for constructing modelling datasets from large raw sources through joins, aggregations, and feature engineering.

Distributed computation does not change statistical principles. Leakage, invalid population definitions, inappropriate aggregation, and temporal contamination remain analytical errors even when the Spark computation itself is technically correct.

Effective PySpark programming therefore requires reasoning simultaneously about:

```
1 analytical correctness
2   +
3 distributed execution
```

A useful mental model is to ask first **what transformation is required**, and then **what data must move in order for Spark to perform it**.

118 Problems

The following problems review the main concepts introduced in this chapter. They progress from basic Spark DataFrame operations to distributed execution, joins, partitioning, performance reasoning, Spark SQL, window functions, storage, and machine learning workflows.

118.1 Problem 1 — Creating a Spark Session

Create a local Spark session named:

```
1 PySparkProblems
```

using all available local processor cores.

Store the resulting object in:

```
1 spark
```

Then inspect:

```
1 spark.version
```

Explain the role of the SparkSession in a PySpark application.

118.2 Problem 2 — Creating a DataFrame

Create a Spark DataFrame from:

```
1 data = [  
2     (1, "Auckland", 120.0),  
3     (2, "Wellington", 95.0),  
4     (3, "Christchurch", 140.0),  
5     (4, "Auckland", 110.0)  
6 ]
```

with columns:

```
1 customer_id  
2 city  
3 sales
```

Display the DataFrame and inspect its schema.

Explain why a Spark DataFrame is conceptually different from a pandas DataFrame even though both appear tabular.

118.3 Problem 3 — Explicit Schema

Recreate the DataFrame from Problem 2 using an explicit schema with:

```
1 customer_id IntegerType  
2 city      StringType  
3 sales     DoubleType
```

Mark customer_id as non-nullable.

Explain one advantage of explicitly defining the schema rather than relying on inference.

118.4 Problem 4 — Basic Inspection

Using the DataFrame from Problem 2, inspect:

```
1 df.columns
2 df.schema
3 df.printSchema()
4 df.show()
```

Then execute:

```
1 df.count()
```

Which of these operations are metadata inspection operations and which require Spark to execute a job?

Explain why this distinction matters for large datasets.

118.5 Problem 5 — Selecting Columns

Select only:

```
1 customer_id
2 sales
```

using:

```
1 select()
```

Then repeat the selection using:

```
1 F.col()
```

Explain what:

```
1 F.col("sales")
```

represents.

Does it retrieve the entire column into the Python driver?

118.6 Problem 6 — Column Expressions

Using the DataFrame from Problem 2, create:

```
1 sales_adjusted
```

where:

$$\text{sales adjusted} = 1.15 \times \text{sales}.$$

Use:

```
1 withColumn()
```

and a Spark column expression.

Explain why no explicit Python loop is required.

118.7 Problem 7 — Multiple Derived Columns

Create:

```
1 tax
2 sales_with_tax
```

where:

$$\text{tax} = 0.15 \times \text{sales},$$

and:

$$\text{sales with tax} = \text{sales} + \text{tax}.$$

Display the final result.

118.8 Problem 8 — Filtering Rows

Select observations where:

$\text{sales} > 100.$

Then select rows where:

$\text{sales} > 100$

and:

$\text{city} = \text{Auckland}.$

Use Spark column expressions.

Explain why parentheses are important when combining conditions.

118.9 Problem 9 — Conditional Columns

Create a column:

```
1 sales_category
```

with values:

```
1 high
2 medium
3 low
```

according to:

high if $\text{sales} \geq 120,$

medium if $100 \leq \text{sales} < 120,$

and:

low otherwise.

Use:

```
1 F.when()
```

and:

```
1 otherwise()
```

118.10 Problem 10 — String Operations

Create a DataFrame containing city names with:

- inconsistent capitalization;
- leading whitespace;
- trailing whitespace.

Use built-in Spark functions to create:

```
1 city_clean
```

with trimmed and consistently formatted values.

Do not use a Python UDF.

Explain why Spark built-in functions are preferable here.

118.11 Problem 11 — Missing Values

Create a DataFrame containing missing values in:

```
1 city  
2 sales
```

Then:

1. identify rows with missing sales;
2. remove rows where sales is missing;
3. replace missing city values with "unknown";

4. replace missing sales values with 0.0.

Explain why missing-value treatment remains an analytical decision even in a distributed system.

118.12 Problem 12 — Duplicates

Create a DataFrame containing several duplicate rows.

Compare:

```
1 distinct()
```

with:

```
1 dropDuplicates()
```

Then remove duplicates based only on:

```
1 customer_id
```

Explain why duplicate detection depends on the expected grain of the dataset.

118.13 Problem 13 — Sorting

Sort observations:

1. by sales ascending;
2. by sales descending;
3. by city ascending and sales descending.

Explain why a global:

```
1 orderBy()
```

can be substantially more expensive in Spark than sorting a small local DataFrame.

118.14 Problem 14 — Global Aggregation

Using a DataFrame containing sales values, calculate:

```
1 count
2 sum
3 mean
4 minimum
5 maximum
6 standard deviation
```

in one aggregation.

Explain conceptually how a distributed sum can be decomposed into partial sums across partitions.

118.15 Problem 15 — Grouped Aggregation

Given:

```
1 data = [  
2     ("Auckland", 120.0),  
3     ("Auckland", 140.0),  
4     ("Wellington", 90.0),  
5     ("Wellington", 110.0),  
6     ("Christchurch", 80.0)  
7 ]
```

calculate for each city:

```
1 n  
2 total_sales  
3 mean_sales  
4 maximum_sale
```

using:

```
1 groupBy()
```

and:

```
1 agg()
```

118.16 Problem 16 — Grain after Aggregation

Suppose the original DataFrame has grain:

```
1 one row per transaction
```

and you execute:

```
1 df.groupBy(  
2   "customer_id"  
3 ).agg(  
4   F.sum("amount")  
5 )
```

What is the grain of the resulting DataFrame?

Explain why aggregation changes the meaning of one row.

118.17 Problem 17 — Transformations and Actions

Classify each operation as primarily a transformation or an action:

```
1 select()  
2 filter()  
3 withColumn()  
4 groupBy()  
5 join()  
6 count()  
7 show()  
8 collect()  
9 take()
```

Then explain why the distinction is important for lazy evaluation.

118.18 Problem 18 — Lazy Evaluation

Consider:

```

1 result = (
2     df
3     .filter(
4         F.col("sales") > 100
5     )
6     .select(
7         "customer_id",
8         "sales"
9     )
10 )

```

Before any action is called:

1. Has Spark necessarily scanned the complete dataset?
2. What has Spark constructed instead?

Then explain what changes when:

```

1 result.count()

```

is executed.

118.19 Problem 19 — Reading an Execution Pipeline

Consider:

```

1 result = (
2     df
3     .filter(
4         F.col("amount") > 0
5     )
6     .withColumn(
7         "revenue",
8         F.col("quantity")
9         * F.col("unit_price")
10    )
11    .groupBy(
12        "customer_id"
13    )
14    .agg(

```

```

15     F.sum("revenue").alias(
16         "total_revenue"
17     )
18 )
19 )

```

Identify:

- the likely narrow transformations;
- the operation most likely to introduce a shuffle;
- the final grain.

Explain your reasoning.

118.20 Problem 20 — Narrow versus Wide Transformations

For each operation below, classify it as typically narrow or potentially wide:

```

1 select
2 filter
3 withColumn
4 groupBy
5 orderBy
6 distinct
7 repartition
8 join

```

Some operations may depend on the situation.

Explain why the narrow/wide distinction is useful for performance reasoning.

118.21 Problem 21 — Shuffle Reasoning

Suppose the following rows exist in different partitions:

```

1 Partition 1
2 Auckland    100
3 Wellington  80
4
5 Partition 2

```

```
6 Auckland 120
7 Christchurch 90
```

You execute:

```
1 df.groupBy(
2   "city"
3 ).sum(
4   "sales"
5 )
```

Explain why Spark may need to move data across partitions.

Which Auckland observations must logically be combined?

118.22 Problem 22 — Stages

Consider the conceptual execution:

```
1 read
2 |
3 filter
4 |
5 withColumn
6 |
7 groupBy
8 |
9 aggregate
```

Explain how Spark could divide this computation into stages.

What operation is likely to create the stage boundary?

118.23 Problem 23 — Basic Join

Create:

```
1 customers
```

with grain:

```
1 one row per customer
```

and:

```
1 orders
```

with grain:

```
1 one row per order.
```

Join them on:

```
1 customer_id
```

and return:

```
1 order_id
2 customer_id
3 customer_name
4 amount
```

Determine the grain of the resulting DataFrame.

118.24 Problem 24 — Join Types

Modify the previous data so that:

- one customer has no orders;
- one order has no matching customer.

Compare:

```
1 inner
2 left
3 right
4 full
```

joins.

Explain which population is preserved in each case.

118.25 Problem 25 — Semi Join

Using the customer and order DataFrames, return only customers that have at least one order.

Use:

```
1 left_semi
```

Explain why a semi join can be preferable to performing a full join and subsequently selecting only customer columns.

118.26 Problem 26 — Anti Join

Return customers having no orders using:

```
1 left_anti
```

Explain the business question represented by the operation.

118.27 Problem 27 — Join Cardinality

Suppose:

```
1 customers
2   one row per customer
3
4 orders
5   many rows per customer
```

Determine the cardinality of:

```
1 customers → orders
```

and explain why joining customer attributes into the order table should not normally increase the number of orders.

118.28 Problem 28 — Many-to-Many Row Multiplication

Suppose key A occurs:

```
1 2 times
```

in the left DataFrame and:

```
1 3 times
```

in the right DataFrame.

How many matching rows will the join produce for A?

Explain why accidental many-to-many joins can be especially dangerous at Spark scale.

118.29 Problem 29 — Join and Shuffle

Consider two very large DataFrames:

```
1 transactions
2 customers
```

both partitioned independently.

They are joined on:

```
1 customer_id
```

Explain why Spark may need to redistribute records before completing the join.

What is being achieved by this shuffle?

118.30 Problem 30 — Broadcast Join

Suppose:

```
1 transactions
2   500 million rows
3
4 countries
5   200 rows
```

and both contain:

```
1 country_code.
```

Would you consider broadcasting countries?

Write the PySpark join using:

```
1 F.broadcast()
```

and explain why this strategy can reduce data movement.

118.31 Problem 31 — When Not to Broadcast

Suppose both sides of a join contain hundreds of millions of observations.

Explain why broadcasting either side may be inappropriate.

What resource constraint becomes important?

118.32 Problem 32 — Logical Join versus Physical Strategy

Explain the distinction between:

```
1 left join
```

and:

```
1 broadcast hash join
```

Why do these describe different aspects of the computation?

118.33 Problem 33 — Number of Partitions

Create a Spark DataFrame and inspect:

```
1 df.rdd.getNumPartitions()
```

Then use:

```
1 repartition()
```

to change the partition count.

Explain why partitions are closely related to available parallelism.

118.34 Problem 34 — `repartition()` versus `coalesce()`

Suppose a DataFrame contains:

```
1 200 partitions
```

and should be reduced to:

```
1 20 partitions.
```

Compare using:

```
1 repartition(20)
```

with:

```
1 coalesce(20)
```

Explain the conceptual difference between the two operations.

118.35 Problem 35 — Repartitioning by Key

Suppose several later operations group by:

```
1 customer_id.
```

Construct:

```
1 df.repartition(  
2     100,  
3     "customer_id"  
4 )
```

Explain why key-based repartitioning might help some downstream operations.

Also explain why it should not be applied automatically.

118.36 Problem 36 — Too Few Partitions

Suppose:

- 1 1 TB dataset
- 2 8 partitions
- 3 100 available executor cores

Discuss the likely problem.

Why may the cluster be underutilised?

118.37 Problem 37 — Too Many Partitions

Suppose:

- 1 10 GB dataset
- 2 1,000,000 partitions

Discuss the likely problems.

Consider:

- scheduling overhead;
- task startup;
- small files;
- metadata overhead.

118.38 Problem 38 — Data Skew

Suppose transaction counts per customer are:

- 1 most customers:
- 2 10–100 transactions
- 3
- 4 customer X:
- 5 50,000,000 transactions

The data is grouped by:

1 `customer_id.`

Explain why this can create skew.

What execution symptom would you expect to observe?

118.39 Problem 39 — Straggler Task

A Spark stage contains 100 tasks.

Ninety-nine finish in:

1 `20 seconds`

while one takes:

1 `15 minutes.`

What might this indicate?

Name at least three plausible causes.

118.40 Problem 40 — More Executors

Suppose a Spark job takes:

1 `30 minutes`

using 10 executors.

Would using 20 executors necessarily reduce runtime to 15 minutes?

Explain why not.

Discuss:

- communication;
- shuffle;
- sequential stages;
- skew;
- scheduling overhead.

118.41 Problem 41 — Repeated Computation

Consider:

```
1 result = (  
2     df  
3     .filter(...)  
4     .join(...)  
5     .groupBy(...)  
6     .agg(...)  
7 )  
8  
9 result.count()  
10 result.show()
```

Explain why the upstream computation may execute more than once.

How could caching change this behaviour?

118.42 Problem 42 — Cache

Persist an expensive DataFrame using:

```
1 cache()
```

Then execute two actions.

Finally remove it using:

```
1 unpersist()
```

Explain when caching is useful.

118.43 Problem 43 — When Not to Cache

For each case, decide whether caching is likely justified:

1. A DataFrame is used once.
2. An expensive join result is used by five downstream analyses.
3. A tiny transformation can be recomputed almost instantly.
4. A very large DataFrame consumes nearly all executor memory.

Justify each answer.

118.44 Problem 44 — collect()

Suppose:

```
1 df
```

contains:

```
1 500 million rows.
```

What happens conceptually when:

```
1 df.collect()
```

is called?

Where do the resulting records go?

Explain why the operation can fail even though Spark was able to process the distributed dataset successfully.

118.45 Problem 45 — toPandas()

Suppose Spark produces a final result containing:

```
1 2,000 rows
2 20 columns.
```

Would:

```
1 df.toPandas()
```

be reasonable?

Now suppose the result contains:

```
1 200 million rows.
```

Explain why the answer changes.

118.46 Problem 46 — Reading an Execution Plan

Create a grouped aggregation and execute:

```
1 result.explain(  
2     mode="formatted"  
3 )
```

Identify any operators corresponding conceptually to:

```
1 scan  
2 filter  
3 aggregate  
4 exchange
```

Explain what an Exchange generally indicates.

118.47 Problem 47 — Filter Early

Consider:

```
1 transactions.join(  
2     products,  
3     "product_id"  
4 ).filter(  
5     F.col("country")  
6     == "New Zealand"  
7 )
```

Suppose the country filter logically applies only to the transaction table.

Rewrite the computation so that filtering occurs before the join.

Explain why reducing data volume before an expensive join can improve performance.

118.48 Problem 48 — Column Pruning

Suppose a source contains:

```
1 100 columns
```

but only:

```
1 customer_id
2 product_id
3 amount
```

are required.

Construct a selection keeping only the required columns.

Explain how carrying fewer columns through downstream stages can reduce computational cost.

118.49 Problem 49 — Built-In Function versus UDF

You need to convert:

```
1 city
```

to uppercase.

Compare:

```
1 F.upper(
2     F.col("city")
3 )
```

with a custom Python UDF performing the same operation.

Which would you prefer?

Explain why.

118.50 Problem 50 — Appropriate Use of a Python UDF

Give an example of a transformation that might genuinely require custom Python logic not available through Spark's built-in expression system.

Explain why the existence of UDF support does not imply that UDFs should be the default solution.

118.51 Problem 51 — Spark SQL

Create a temporary view:

```
1 sales
```

and write a SQL query calculating total sales by city.

Then implement the same computation using the DataFrame API.

Compare the two versions.

Do they necessarily use fundamentally different execution engines?

118.52 Problem 52 — SQL Join

Using temporary views:

```
1 customers
2 orders
```

write a SQL query performing a left join and returning:

```
1 order_id
2 customer_name
3 amount
```

Load the result as a Spark DataFrame.

118.53 Problem 53 — Window versus Aggregation

Suppose a transaction DataFrame has grain:

```
1 one row per transaction.
```

Calculate:

```
1 customer_total
```

for every transaction using a window.

Then calculate total sales using:

```
1  groupBy(  
2    "customer_id"  
3  )
```

Compare the grain of the two outputs.

118.54 Problem 54 — Row Number

Given:

```
1  customer_id  
2  order_date
```

number each customer's orders chronologically starting from 1.

Use:

```
1  row_number()
```

and an appropriate window.

118.55 Problem 55 — Ranking

Rank transactions by sales within each city.

Compare:

```
1  rank()
```

and:

```
1  dense_rank()
```

when ties occur.

118.56 Problem 56 — Lag

For each customer, order observations by date and create:

```
1 previous_amount
2 amount_change
```

using:

```
1 lag()
```

Explain the first observation's result.

118.57 Problem 57 — Running Total

Create:

```
1 cumulative_sales
```

for each customer ordered by date.

Use a window from:

```
1 unbounded preceding
```

to:

```
1 current row.
```

Explain the mathematical meaning of the resulting column.

118.58 Problem 58 — Window Performance

Why can:

```
1 partitionBy(
2     "customer_id"
3 ).orderBy(
4     "date"
5 )
```

require substantial computation?

Discuss both:

- partition redistribution;
- sorting.

118.59 Problem 59 — Writing Parquet

Write a DataFrame to:

```
1 output/sales
```

using Parquet and overwrite mode.

Then describe the likely physical output structure.

Why should you expect a directory containing multiple files rather than one single file?

118.60 Problem 60 — CSV versus Parquet

Compare CSV and Parquet for a large analytical dataset.

Discuss:

- schema;
- columnar storage;
- compression;
- column pruning;
- human readability;
- interoperability.

Which format would you generally prefer for repeated large-scale analytical processing?

118.61 Problem 61 — Partitioned Output

Suppose transactions contain:

```
1 year
2 month
```

Write them using:

```
1 partitionBy(  
2   "year",  
3   "month"  
4 )
```

Explain how this may help later queries filtering by time.

118.62 Problem 62 — Bad Partition Column

Would:

```
1 transaction_id
```

typically be a good physical partition column if every transaction has a unique ID?

Explain why not.

118.63 Problem 63 — Small Files Problem

Suppose a Spark pipeline produces:

```
1 2 million Parquet files
```

averaging:

```
1 10 KB each.
```

Explain why this can be inefficient even though the total data size is manageable.

118.64 Problem 64 — Arrow

Explain the role of Apache Arrow when converting between Spark and pandas.

Does Arrow make:

```
1 df.toPandas()
```

safe for arbitrarily large DataFrames?

Explain why not.

118.65 Problem 65 — Pandas UDF

Write a simple Pandas UDF that receives a pandas Series and returns:

$$x - \bar{x}.$$

Explain how this differs conceptually from a traditional row-by-row Python UDF.

118.66 Problem 66 — PySpark or pandas?

For each workload, choose PySpark or pandas as the more natural starting point.

1. A 200 MB CSV explored interactively on a laptop.
2. A 15 TB transaction dataset stored in distributed object storage.
3. A final aggregated table containing 10,000 rows.
4. Joining several multi-terabyte data-lake tables.
5. Creating a small publication-quality visualization.
6. Calculating simple summary statistics from a dataset already loaded into pandas.

Justify each choice.

118.67 Problem 67 — Building a Modelling Table

Suppose raw data consists of:

```
1 customers
2   one row per customer
3
4 orders
5   one row per order
6
7 order_lines
8   one row per product per order
```

You need a modelling table containing one row per customer with:

```
1 number_of_orders
2 total_units
3 total_spending
```

Describe the Spark transformations required.

At every intermediate step, state the grain.

118.68 Problem 68 — Leakage in Distributed Feature Engineering

Suppose churn is predicted on January 1, 2026.

A PySpark aggregation calculates:

```
1 customer total spending
```

using all transactions through December 31, 2026.

Why is this feature invalid?

Does Spark's distributed correctness prevent this mistake?

118.69 Problem 69 — VectorAssembler

Given columns:

```
1 age
2 income
3 tenure
```

use:

```
1 VectorAssembler
```

to create:

```
1 features
```

Explain why many Spark ML algorithms expect predictors in a vector column.

118.70 Problem 70 — Estimator and Model

Consider:

```

1 regression = LinearRegression(
2     featuresCol="features",
3     labelCol="target"
4 )
5
6 model = regression.fit(
7     training_data
8 )

```

Identify:

```

1 Estimator
2 Model
3 fit()
4 learned state

```

Explain the lifecycle conceptually.

118.71 Problem 71 — Spark ML Pipeline

Create a simple Spark ML pipeline containing:

```

1 VectorAssembler
2 LinearRegression

```

Fit it on training data.

Then apply the fitted pipeline to test data.

Explain what happens to the estimator stage during:

```

1 pipeline.fit(...)

```

118.72 Problem 72 — Distributed Data Preparation versus Distributed Training

Suppose raw data contains:

```

1 10 TB

```

but after filtering and aggregation the final modelling dataset contains:

```
1 500,000 rows.
```

Discuss whether the model necessarily needs to be trained using Spark ML.

Propose an alternative architecture.

118.73 Problem 73 — Diagnose a Poor Spark Workflow

Consider:

```
1 df = spark.read.parquet(  
2     "transactions"  
3 )  
4  
5 rows = df.collect()  
6  
7 result = []  
8  
9 for row in rows:  
10     ...
```

Critique the design.

Why does it largely defeat the purpose of Spark?

Propose a distributed alternative.

118.74 Problem 74 — Diagnose Repeated Actions

Consider:

```
1 print(df.count())  
2 print(df.filter(...).count())  
3 print(df.groupBy(...).count())  
4 print(df.count())
```

Discuss how many distributed computations may be triggered.

Would caching df necessarily be useful?

What information would you need before deciding?

118.75 Problem 75 — Diagnose Unnecessary Repartitioning

Consider:

```
1 df = (  
2   df  
3   .repartition(500)  
4   .filter(...)  
5   .repartition(200)  
6   .withColumn(...)  
7   .repartition(1000)  
8 )
```

Critique the pipeline.

Why can repeated repartitioning be expensive?

118.76 Problem 76 — Diagnose a Join Strategy

Suppose:

```
1 events  
2   5 TB  
3  
4 country_codes  
5   20 KB
```

and the two datasets are joined by:

```
1 country_code.
```

A physical plan shows both sides being shuffled.

What alternative strategy would you investigate?

What would you verify before forcing it?

118.77 Problem 77 — Diagnose Skew

Suppose a join stage contains:

1 500 tasks

with most processing:

1 100 MB

but several processing:

1 30 GB.

Explain the likely issue.

What characteristic of the join key might cause this?

118.78 Problem 78 — Execution Plan Reasoning

Suppose an execution plan conceptually contains:

```
1 Scan
2 |
3 Filter
4 |
5 Project
6 |
7 Exchange
8 |
9 HashAggregate
```

Explain what each stage represents.

Which operation most clearly signals data redistribution?

118.79 Problem 79 — End-to-End Spark Reasoning

Consider the following pipeline:

```

1 result = (
2   transactions
3   .select(
4     "customer_id",
5     "product_id",
6     "amount",
7     "date"
8   )
9   .filter(
10    F.col("amount") > 0
11  )
12  .join(
13    F.broadcast(products),
14    on="product_id",
15    how="left"
16  )
17  .groupBy(
18    "customer_id"
19  )
20  .agg(
21    F.sum("amount").alias(
22      "total_amount"
23    ),
24    F.countDistinct(
25      "product_id"
26    ).alias(
27      "n_products"
28    )
29  )
30  .cache()
31 )

```

Without executing the code, explain:

1. the initial grain;
2. why only selected columns are retained;
3. whether the filter is likely narrow;
4. why products might be broadcast;
5. whether the join necessarily preserves grain;
6. why groupBy() can introduce a shuffle;

7. the grain after aggregation;
8. why `countDistinct()` may be more expensive than ordinary counting;
9. when caching might be justified;
10. what action is required before the cache is actually populated.

118.80 Problem 80 — Complete Architecture Exercise

Suppose an organisation stores:

```
1 8 TB of transaction data per year
```

in Parquet files.

The objective is to construct a monthly customer-level dataset for a churn model.

The source contains:

```
1 transactions
2 customers
3 products
```

and the final modelling dataset should contain:

```
1 customer_id
2 month
3 transaction_count
4 total_spending
5 average_spending
6 number_of_products
7 customer_age
8 region
```

Design the PySpark workflow.

Your answer should address:

1. which columns should be read;
2. which filters should be applied early;
3. the grain of every input;
4. expected join cardinalities;
5. whether any lookup table could potentially be broadcast;
6. which operations are likely to cause shuffles;

7. where data skew might arise;
8. the grain after aggregation;
9. whether caching is required;
10. an appropriate output format;
11. an appropriate physical partitioning strategy;
12. whether the final dataset might later be moved to pandas;
13. whether model training necessarily needs Spark ML;
14. how temporal leakage should be prevented;
15. which parts of the execution plan you would inspect if performance were poor.

Finally, summarise the architecture conceptually as:



The objective is to design a workflow that is both analytically correct and computationally appropriate for distributed execution.